

EMSI — École Marocaine des Sciences de l'Ingénieur
(Casablanca)

Rapport de Projet de Fin d'Année

NEXORA

Plateforme d'Intelligence Organisationnelle

Réalisé par : Soubaïbi Omar
Yassine Baïzi

Encadré par : Mme. Ounacer Soumaya

Filière : 4IADM G5-A

Année Universitaire : 2025–2026

Dédicace

Nous dédions ce travail à nos familles, dont le soutien constant, la patience et les encouragements ont constitué un socle indispensable tout au long de ce projet. Leur confiance nous a permis de maintenir l'exigence et la persévérance nécessaires à la réalisation d'un système ambitieux, à la croisée de l'intelligence artificielle, du Big Data et de l'ingénierie logicielle.

Nous dédions également ce rapport à toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de NEXORA, que ce soit par un conseil, une relecture, un retour critique ou un soutien moral. Chaque interaction a participé à renforcer la rigueur de la démarche et la qualité du résultat.

Enfin, nous dédions ce travail à notre binôme, pour l'engagement, la complémentarité et la discipline de travail. La construction d'une plateforme cohérente, industrialisable et documentée a été rendue possible par une collaboration constante, structurée et orientée vers l'excellence.

Remerciements

Nous tenons à exprimer notre profonde gratitude à **Mme. Ounacer Soumaya**, encadrante du projet, pour la qualité de son suivi, sa disponibilité et la pertinence de ses conseils. Son accompagnement méthodologique et ses orientations techniques ont contribué à structurer la démarche, à renforcer l'exigence scientifique et à assurer la cohérence globale du système développé.

Nous remercions également l'**équipe pédagogique de la filière 4IADM (EMSI)** pour l'ensemble des enseignements dispensés en intelligence artificielle, Big Data, bases de données et ingénierie logicielle. Les compétences acquises au cours de la formation ont constitué le socle nécessaire pour concevoir une solution complète, intégrant à la fois un graphe de connaissances, des algorithmes d'analyse et une architecture de déploiement reproductible.

Nous adressons aussi nos remerciements à la **direction de l'EMSI Casablanca**, pour le cadre académique d'excellence mis à disposition, la mise en place d'un environnement stimulant et l'encouragement constant à la réalisation de projets à forte valeur technologique.

Enfin, nous exprimons notre reconnaissance à nos **familles et proches**, pour leur soutien moral, leur patience et leurs encouragements continus.

Résumé

Résumé

Dans les grandes organisations, la connaissance des compétences internes demeure fréquemment fragmentée par des silos d'information, rendant difficile l'identification rapide d'experts, l'anticipation des besoins en compétences et la collaboration interdépartementale. Cette fragmentation se traduit par des coûts opérationnels (temps de recherche, redondances), une sous-utilisation des talents disponibles et une difficulté à capitaliser sur la production documentaire.

Pour répondre à ces enjeux, le projet **NEXORA** propose une plateforme d'**intelligence organisationnelle** combinant un **graphe de connaissances Neo4j**, des modules d'**apprentissage automatique** (PageRank, TF-IDF, filtrage collaboratif, classification) et une couche **Big Data** (PySpark en batch, pipeline streaming via simulateur Kafka). La solution permet la cartographie interactive 3D des entités (experts, compétences, projets, documents), la recherche d'experts par pertinence sémantique et l'explicabilité par relations. Elle offre également des recommandations (experts, compétences) ainsi qu'un chatbot IA (**Veda**) exploitant un LLM (Groq) avec repli rule-based afin de maintenir un service fonctionnel en cas d'indisponibilité externe.

Sur le plan architectural, le système adopte une approche **API-First** avec FastAPI et une interface **SPA** (React 18 + TypeScript + MUI), le tout déployé via **Docker Compose** derrière Nginx. La sécurité est assurée par **JWT + RBAC** (expiration configurable, gardes serveur) et des mécanismes de robustesse sont présents (fallback JSONL/analytics in-memory lorsque nécessaire). Le backend expose des endpoints IA/Big Data, des fonctionnalités collaboratives (WebSocket) et une documentation interactive (Swagger).

Dans sa version finale, le projet est **finalisé**, totalisant **17 200+ lignes** et environ **100 fichiers**, avec **17 pages frontend**, **19 routers backend** et **9 modules ML**. Les données sont fournies au format **JSONL** (employees, skills, projects, documents, ainsi que des catalogues additionnels : companies et technologies), afin de faciliter le chargement, les traitements batch et les scénarios de tests à différentes échelles.

Mots-clés : Intelligence organisationnelle, graphe de connaissances, Neo4j, apprentissage automatique, Big Data, FastAPI, React.

Abstract

In large organizations, internal skills knowledge is often fragmented across information silos, which makes it difficult to quickly identify experts, anticipate skill gaps, and enable cross-department collaboration. This fragmentation leads to duplicated efforts, longer staffing cycles, and limited knowledge reuse.

To address these challenges, **NEXORA** delivers an **organizational intelligence** platform combining a **Neo4j knowledge graph**, **machine learning modules** (PageRank, TF-IDF, collaborative filtering, classification), and

a **Big Data layer** (PySpark batch processing and streaming via a Kafka simulator). The solution provides an interactive 3D visualization of entities (experts, skills, projects, documents), semantic expert search with relationship-based explainability, AI-driven recommendations, and an AI chatbot (**Veda**) backed by a Groq LLM with a rule-based fallback to remain usable even when external LLM access is unavailable.

The system follows an **API-first** architecture (FastAPI) and a **SPA** frontend (React 18 + TypeScript + MUI), deployed through **Docker Compose** behind Nginx. Security relies on **JWT + RBAC** (configurable token expiration and server-side guards). Robustness mechanisms are included (JSONL fallbacks and in-memory analytics when batch outputs are missing), and real-time collaboration features leverage WebSockets.

In its final version, the project is **completed**, totaling **17,200+ lines of code** across around **100 source files**, with **17 frontend pages**, **19 backend routers**, and **9 ML modules**. Data is provided in **JSONL** (employees, skills, projects, documents, plus additional catalogs such as companies and technologies) to support reproducible datasets, batch processing, and scalable experiments.

Keywords : Organizational intelligence, knowledge graph, Neo4j, machine learning, Big Data, FastAPI, React.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
1 Introduction Générale	1
1.1 Contexte Général	1
1.2 Problématique	2
1.3 Objectifs du Projet	2
1.4 Méthodologie de Travail	3
1.5 Plan du Rapport	3
1.6 Technologies Utilisées	4
2 État de l'Art	5
2.1 Introduction	5
2.2 Les Bases de Données Orientées Graphe	6
2.3 Frameworks Web Modernes	7
2.4 Solutions de Gestion des Connaissances Existantes	8
2.5 Positionnement de NEXORA	8
3 Architecture du Système	10
3.1 Vue d'Ensemble	10
3.2 Structure du Projet	12
3.3 Communication entre Couches	13
3.4 Patterns de Conception Utilisés	14
3.5 Flux de Données Complet	16
3.6 Gestion des Erreurs	18

4	Gestion de Projet	19
4.1	Objectifs et Périmètre	19
4.2	Charte de Projet (Cadrage)	19
4.3	Cahier des Charges (Expression des Besoins)	20
4.4	Découpage (WBS) et Lots	20
4.5	Planification : Diagramme de Gantt et Chemin Critique	21
4.5.1	Diagramme de Gantt	21
4.5.2	Chemin Critique (méthode CPM)	23
4.5.3	Tableau PERT : tâches, durées, dépendances et marges	24
4.6	Méthodologie Itérative et Backlog Produit	24
4.6.1	Suivi d'avancement par sprint	25
4.7	Matrice RACI	26
4.8	Matrice des Risques	27
4.9	Indicateurs de Suivi (KPI)	27
4.10	Synthèse de Gestion de Projet	28
5	Études Stratégiques et de Faisabilité	29
5.1	Analyse PESTEL	29
5.2	Analyse SWOT	30
5.3	Analyse des 5 Forces de Porter	31
5.4	Analyse Croisée et Recommandations Stratégiques	31
5.4.1	Business Model Canvas	32
5.5	Étude Technique	33
5.5.1	Besoins Matériels	33
5.5.2	Ressources Humaines	33
5.5.3	Logiciels et Services	34
5.6	Étude Commerciale et Marketing	34
5.6.1	Outils et services acquis	34
5.7	Étude Financière Simplifiée	35
6	Conception et Implémentation Technique	36
6.1	Diagrammes UML	36
6.1.1	Architecture 3-tiers	36
6.1.2	Diagramme de cas d'usage	38
6.1.3	Diagramme de classes	41
6.1.4	Diagramme de séquence — Authentification	43
6.1.5	Schéma de déploiement	44
6.1.6	Schéma du graphe de données	45
6.1.7	Modèle de données	46
6.1.8	Pipeline d'analyse ML	47

6.2	Base de Données Neo4j	48
6.3	Pourquoi Neo4j ?	48
6.4	Schéma de Données	48
6.5	Statistiques de la Base de Données	49
6.6	Backend FastAPI	49
6.7	Présentation de FastAPI	49
6.8	Endpoints API	51
6.9	Authentification JWT	52
6.10	Frontend React	53
6.11	Architecture du Frontend	53
6.12	Visualisation 3D	54
6.13	Présentation de la Plateforme	56
6.13.1	Page de Connexion	56
6.13.2	Recherche d'Experts	56
6.13.3	Chatbot Veda	57
6.13.4	Documentation API (Swagger)	58
6.13.5	Autres Interfaces	58
6.14	Conclusion	62
7	Données et Déploiement	63
7.1	Guide d'Installation	63
7.2	Prérequis	63
7.3	Variables d'Environnement	64
7.4	Données du Système	64
7.5	Format des Données	64
7.6	Sécurité et Performance	69
7.6.1	Authentification et Autorisation	69
8	Conclusion et Perspectives	71
8.1	Bilan du Projet	71
8.2	Points Forts	72
8.3	Perspectives d'Amélioration	73
8.4	Conclusion Générale	74
A	Guide de Démarrage Rapide	76
	Bibliographie et Webographie	77
B	Glossaire	79

Table des figures

4.1	Diagramme de Gantt de NEXORA avec chemin critique (rouge) et dépendances	22
6.1	Architecture 3-tiers de NEXORA	36
6.2	Cas d’usage principaux de NEXORA	38
6.3	Diagramme de classes (vue de conception)	41
6.4	Séquence d’authentification : login, génération et validation JWT	43
6.5	Schéma de déploiement (Docker Compose / services)	44
6.6	Schéma et relations clés du graphe (représentation)	45
6.7	Modèle de données (vue applicative et entités principales)	46
6.8	Pipeline d’analyse et modules IA (vue d’ensemble)	47
6.9	Documentation interactive de l’API (Swagger / OpenAPI)	50
6.10	Documentation interactive de l’API (Swagger / OpenAPI)	58
6.11	Fil d’actualité (Feed)	58
6.12	Page de profil utilisateur	59
6.13	Tableau de bord — vue complémentaire	59
6.14	Réseau de connexions	60
6.15	Offres et opportunités	60
6.16	Cartographie des compétences (Skill Map)	61
6.17	Analyse des lacunes de compétences (Skill Gap)	61
6.18	Évolution des compétences	62

Liste des tableaux

1.1	Récapitulatif des technologies de NEXORA	4
2.1	Comparaison : Base Relationnelle vs Base Graphe	7
4.1	WBS (Work Breakdown Structure) spécifique à NEXORA	21
4.2	Analyse PERT — calcul des dates et marges (en semaines)	24
4.3	Backlog produit organisé par sprints (priorité MoSCoW)	25
4.4	Tableau de bord d'avancement par sprint	25
4.5	Matrice RACI détaillée pour NEXORA	26
4.6	Matrice des risques et plans de mitigation (NEXORA)	27
4.7	Tableau de bord des KPI du projet NEXORA	28
4.8	Synthèse des pratiques de gestion de projet appliquées	28
5.1	Analyse PESTEL de NEXORA	29
5.2	Analyse SWOT de NEXORA	30
5.3	Analyse des 5 forces de Porter pour NEXORA	31
5.4	Matrice de recommandations stratégiques (croisement SWOT-PESTEL)	31
5.5	Business Model Canvas de NEXORA	32
5.6	Matériels informatiques nécessaires	33
5.7	Ressources humaines nécessaires	33
5.8	Logiciels et services	34
5.9	Stratégie marketing (4P) pour NEXORA	34
5.10	Budget de prospection et ventes (semestriel)	35
6.1	Types de nœuds dans Neo4j	49
6.2	Statistiques de la base (ordres de grandeur)	49
6.3	Endpoints d'authentification	51
7.1	Variables d'environnement principales	64
7.2	Fichiers de données JSONL	66
7.3	Matrice RBAC (synthèse)	70
8.1	Synthèse des objectifs et état d'implémentation	72

Chapitre 1

Introduction Générale

1.1 Contexte Général

Dans les grandes organisations, l'information relative aux compétences et aux expertises internes est fréquemment disséminée dans des systèmes hétérogènes : gestion des ressources humaines, outils de gestion de projets, bases documentaires, messageries et espaces collaboratifs. Cette fragmentation engendre des silos d'information qui réduisent la capacité à mobiliser rapidement les talents disponibles. En pratique, l'identification de l'expert adéquat dépend souvent de réseaux informels et de connaissances tacites, ce qui ralentit la constitution d'équipes, dégrade l'efficacité opérationnelle et limite la capitalisation.

Cette situation se renforce avec l'augmentation de la volumétrie des données et l'évolution rapide des métiers. Les compétences deviennent transversales, les rôles hybrides se multiplient, et les projets requièrent des interactions entre départements. Une approche reposant uniquement sur des référentiels statiques et des formulaires déclaratifs atteint alors ses limites : elle ne capture pas correctement la dynamique organisationnelle et n'offre pas de mécanismes efficaces de découverte.

Une approche par **graphe de connaissances** s'avère pertinente car elle modélise explicitement les relations entre entités : expert, compétence, projet, document. Les traversées de graphe et les algorithmes de centralité (tels que PageRank) permettent d'extraire des signaux organisationnels, tandis que les approches sémantiques (TF-IDF, similarité cosinus) améliorent la recherche en langage naturel. La section suivante formalise la problématique adressée par NEXORA.

1.2 Problématique

La problématique centrale du projet peut être formulée comme suit : **comment une organisation peut-elle cartographier, valoriser et exploiter ses compétences internes, tout en facilitant l’identification d’experts, la collaboration inter-équipes et l’anticipation des besoins en compétences ?** Cette problématique recouvre des enjeux informationnels (cohérence, intégration), algorithmiques (recommandation, classement) et logiciels (architecture, sécurité, scalabilité).

Un premier sous-problème concerne la **découverte d’expertise**. L’expertise réelle est rarement centralisée et demeure partiellement observée : compétences déclarées, participation aux projets, et contributions documentaires sont souvent dispersées et difficiles à relier. Il devient nécessaire d’établir un modèle de connaissance qui rende la relation calculable et explicable, au lieu de la laisser implicite.

Un second sous-problème est la **recommandation contextualisée**. Proposer des experts pour une requête libre, détecter des lacunes de compétences, ou suggérer des collaborations interdépartementales suppose de combiner plusieurs signaux (profil, relations, contenu, activité). Les solutions existantes peuvent être coûteuses ou opaques. NEXORA vise au contraire une logique explicite, où les résultats sont justifiés par le graphe et les modèles.

Enfin, un troisième sous-problème porte sur la **mise à l’échelle** et la robustesse. Une solution pertinente doit pouvoir traiter des volumes croissants, supporter des traitements batch/streaming et garantir la sécurité des données RH par authentification, autorisation et pratiques de déploiement. La section suivante présente les objectifs opérationnels.

1.3 Objectifs du Projet

Le projet NEXORA poursuit plusieurs objectifs complémentaires. Le premier objectif consiste à **cartographier visuellement les compétences** via un graphe interactif, en fournissant une représentation exploitable par des acteurs RH et techniques. La visualisation 3D, fondée sur WebGL, vise à rendre perceptibles des structures telles que les clusters de compétences et les experts centraux.

Le second objectif est l’**identification d’experts** à l’aide d’algorithmes de centralité, notamment **PageRank**. Dans un graphe organisationnel, l’influence ne se réduit pas à la déclaration ; elle se manifeste par des relations, des contributions et une diversité de compétences. NEXORA exploite ces signaux pour produire des classements pertinents.

Le troisième objectif concerne la **prédiction et la recommandation** : recherche d’experts par requête libre (TF-IDF + cosinus), suggestion de compétences à acquérir (skill gaps) et recommandations personnalisées. Le système inclut également un assistant conver-

sationnel (Veda) qui offre un accès naturel à la connaissance interne via un LLM, tout en conservant un mode de repli rule-based.

Le quatrième objectif porte sur la **démonstration Big Data** : intégration de PySpark (batch) et d'un flux de type streaming (simulateur Kafka). L'objectif est de valider une chaîne complète : ingestion, analytique, exposition API, et restitution dans l'interface. La section suivante décrit la méthodologie de travail.

1.4 Méthodologie de Travail

Le développement de NEXORA a été conduit selon une approche itérative, inspirée de l'agilité. Les fonctionnalités ont été intégrées par incréments : socle API + base de données, interface frontend, puis modules IA (recommandation, classification, centralité), visualisation 3D et enfin aspects de sécurité et de validation.

La réalisation en binôme a permis une parallélisation contrôlée : découpage par couches (frontend, backend, base) et par modules (ML/IA, Big Data). Git a assuré la traçabilité des évolutions et Docker la reproductibilité, réduisant ainsi les écarts d'environnement.

La gestion des risques techniques a pris en compte l'indisponibilité de services (fallback), la stabilité des dépendances et la sécurité (JWT, RBAC, rate limiting). La section suivante introduit le plan du rapport.

1.5 Plan du Rapport

Le rapport est structuré en treize chapitres. Le Chapitre 2 présente l'état de l'art (bases de données graphe, frameworks web, solutions concurrentes). Le Chapitre 3 détaille l'architecture, les patterns et le flux de données.

Le Chapitre 4 est dédié à la gestion de projet (WBS, RACI, diagramme de Gantt, risques). Les Chapitres 5 à 7 décrivent respectivement la base Neo4j, le backend FastAPI et le frontend React. Les Chapitres 8 à 13 abordent les fonctionnalités applicatives, l'installation, les données, la sécurité/performance, la validation par tests, puis la conclusion et les perspectives.

1.6 Technologies Utilisées

TABLE 1.1 – Récapitulatif des technologies de NEXORA

Domaine	Technologies
Frontend	React 18, TypeScript, MUI, Vite, React Router v6, Axios, react-force-graph-3d
Backend	FastAPI, Python 3.11, Uvicorn (ASGI), Pydantic
Base de données	Neo4j 5 Community, APOC, Cypher, Bolt
ML / IA	PageRank, TF-IDF + cosinus, filtrage collaboratif, régression logistique, Groq LLM (Veda), GraphRAG
Big Data	PySpark (batch), simulateur Kafka (streaming)
Déploiement	Docker Compose, Nginx (frontend)
Sécurité	JWT, bcrypt, RBAC (admin/manager/user), rate limiting login
Tests	Pytest, FastAPI TestClient

La Figure ?? fournit un second exemple d'écran d'authentification, utile pour documenter la cohérence UX et le rôle des variables d'environnement liées à la sécurité (clé JWT, durée d'expiration, etc.).

Cette interface illustre plusieurs aspects clés du système d'authentification :

- **Champs de saisie sécurisés** : les champs email et mot de passe sont masqués pour éviter l'affichage en clair des informations sensibles
- **Validation côté client** : les formulaires vérifient le format des entrées avant soumission pour réduire les requêtes invalides
- **Feedback utilisateur** : les messages d'erreur sont affichés clairement en cas d'échec d'authentification
- **Intégration JWT** : après une connexion réussie, le token est stocké et utilisé pour les requêtes ultérieures

Chapitre 2

État de l’Art

2.1 Introduction

Le marché de la gestion des compétences (Skills Management) et de la Talent Intelligence s’est fortement développé, sous l’effet de la transformation numérique et des exigences de mobilité interne. Les organisations cherchent à :

- Réduire les délais de staffing
- Améliorer l’alignement compétences–projets
- Structurer des parcours de formation pilotés par des données

Néanmoins, l’efficacité d’une solution dépend de sa capacité à relier des informations hétérogènes :

- Profils et compétences des employés
- Projets et contributions documentaires
- Interactions et formations

Une approche purement déclarative tend à produire un référentiel statique peu exploitable pour la recommandation et la découverte d’expertise. Il devient alors pertinent d’adopter des modèles capables de représenter des relations riches et évolutives.

Dans ce cadre, les bases de données orientées graphe et les approches hybrides (graph analytics + ML + IA générative) constituent un axe majeur : elles permettent de passer d’une description descriptive à une intelligence calculable et visualisable. La section suivante présente les fondements des bases orientées graphe.

2.2 Les Bases de Données Orientées Graphe

Une base orientée graphe modélise l'information sous forme de nœuds et de relations, chacun pouvant porter des propriétés. Cette représentation est particulièrement adaptée lorsque la relation est centrale :

- Collaboration entre experts
- Co-occurrence de compétences
- Dépendances entre projets
- Liens entre documents

Les requêtes portent sur des motifs relationnels, des chemins et des voisinages.

À l'inverse, une base relationnelle excelle pour des transactions structurées, mais peut devenir coûteuse pour des analyses relationnelles profondes (multi-jointures). Les opérations suivantes sont moins naturelles en SQL et complexifient la maintenance :

- Recherche de chemins
- Détection de communautés
- Traversées successives

Neo4j est pertinent pour NEXORA en raison de :

- **Index-free adjacency** : navigation directe entre nœuds reliés
- **Cypher** : langage déclaratif pour les requêtes graphe
- **Extension APOC** : procédures et fonctions étendues

D'autres solutions existent (Neptune, ArangoDB, TigerGraph), mais Neo4j offre un compromis adapté entre maturité, expressivité et facilité d'intégration. Le tableau suivant synthétise la comparaison.

Cette comparaison met en évidence les avantages spécifiques d'une base de données orientée graphe pour NEXORA :

- **Performance des traversées** : Neo4j utilise l'index-free adjacency pour naviguer directement entre nœuds reliés, sans jointures coûteuses
- **Expressivité des requêtes** : le langage Cypher permet d'exprimer des motifs relationnels complexes de manière naturelle
- **Flexibilité du schéma** : le modèle graphe s'adapte facilement à l'évolution des besoins sans migrations de schéma lourdes
- **Analytics intégrés** : les algorithmes de centralité et de détection de communautés sont natifs

TABLE 2.1 – Comparaison : Base Relationnelle vs Base Graphe

Critère	Relationnelle (SQL)	Graphe (Neo4j)
Modèle	Tables, lignes, jointures	Nœuds, relations, propriétés
Exploration relationnelle	Coûteux (multi-JOIN)	Natif (traversées)
Chemins / motifs	Non naturel	Expressif (Cypher)
Analytics	Externe	Centralité, motifs, voisinage
Cas d’usage	Transactions, reporting	Recommandation, connaissance

2.3 Frameworks Web Modernes

Le choix des frameworks web conditionne la performance, la maintenabilité et la qualité du contrat d’API. FastAPI, retenu dans NEXORA, se distingue par la génération OpenAPI automatique, le typage et la validation (Pydantic), ainsi que la compatibilité ASGI, utile pour les endpoints asynchrones et les WebSocket.

Django constitue un framework robuste, mais plus orienté monolithe et rendu serveur, ce qui peut alourdir une démarche API-First. Express (Node.js) est flexible, mais impose une discipline supplémentaire pour atteindre un niveau équivalent de validation et de documentation. Dans NEXORA, la cohérence du contrat et l’intégration des modules ML (Python) ont motivé l’option FastAPI.

Les avantages spécifiques de FastAPI pour ce projet incluent :

- **Documentation automatique** : OpenAPI/Swagger généré automatiquement pour tous les endpoints
- **Validation stricte** : Pydantic valide les entrées/sorties avec des types Python natifs
- **Performance** : support ASGI pour les opérations asynchrones et WebSocket
- **Intégration ML** : écosystème Python riche (scikit-learn, pandas, numpy)

Côté frontend, React 18 et TypeScript offrent une base solide pour une SPA riche. L’écosystème MUI apporte des composants réutilisables et une cohérence UI, tandis que Vite optimise le build. La visualisation 3D, rendue via WebGL, renforce la capacité d’exploration de la connaissance organisationnelle.

Les choix technologiques pour le frontend reposent sur plusieurs critères :

- **Performance** : React 18 avec automatic batching réduit les re-rendus inutiles
- **Type safety** : TypeScript détecte les erreurs dès la compilation
- **Cohérence UI** : Material-UI fournit des composants prêts à l’emploi et thématiques
- **Vitesse de développement** : Vite accélère le build et le hot reload

- **Visualisation avancée** : WebGL permet le rendu 3D performant dans le navigateur

2.4 Solutions de Gestion des Connaissances Existantes

Les solutions concurrentes proposent des fonctionnalités de cartographie et de recommandation, mais diffèrent sur la transparence des modèles et l'ouverture :

- **365Talents** : capacités IA avancées, mais graphe non explicitable
- **Workday Skills Cloud** : ontologie massive et intégration SI forte, mais écosystème propriétaire
- **Cornerstone OnDemand** : riche en LMS, mais moins orienté cartographie relationnelle
- **LinkedIn Talent Insights** : puissance analytique sur données externes, pas de cartographie interne
- **Skills Base** : accessible mais largement déclaratif, peu orienté IA

Ces constats motivent le positionnement de NEXORA, dont la valeur ajoutée repose sur :

- **Graphe natif explicite** : relations modélisées et navigables pour l'explicabilité
- **Visualisation 3D immersive** : exploration interactive du graphe organisationnel
- **Chaîne IA hybride** : combinaison d'algorithmes classiques et de LLM
- **Architecture ouverte** : API-First et technologies open source
- **Mode dégradé** : fallback JSONL pour continuité de service

2.5 Positionnement de NEXORA

NEXORA combine :

- Un graphe Neo4j pour la modélisation des relations
- Une visualisation 3D WebGL pour l'exploration interactive
- Une chaîne IA hybride (PageRank, TF-IDF + cosinus, filtrage collaboratif, classification, assistant LLM)

Cette combinaison fournit plusieurs modes d'accès à la connaissance :

- Exploration visuelle du graphe
- Recherche sémantique d'experts et documents
- Recommandation personnalisée
- Conversation avec l'assistant Veda

Les différenciateurs clés de NEXORA sont :

- **Centralité du graphe** : relations explicitement modélisées pour calcul de chemins, détection de ponts inter-départements et explicabilité

- **Hybridation ML + LLM** : résultats produits par scores quantitatifs (cosinus, PageRank) et contextualisés via Veda
- **Intégration Big Data** : PySpark + streaming simulé, rarement mis en œuvre dans des projets académiques

Il convient toutefois de noter des limites :

- Le streaming est simulé (pas de cluster Kafka réel)
- Le fallback JSONL doit être gouverné pour éviter des divergences

Ces aspects sont traités comme des défis d'ingénierie dans la suite du rapport.

Chapitre 3

Architecture du Système

3.1 Vue d'Ensemble

NEXORA adopte une architecture 3-tiers :

- **Couche présentation** : React avec TypeScript pour l'interface utilisateur
- **Couche métier** : FastAPI avec Python pour la logique métier et l'API
- **Couche données** : Neo4j (graphe), JSONL (fallback), résultats Spark (batch)

Cette séparation vise à :

- Limiter le couplage entre composants
- Faciliter l'évolution indépendante des couches
- Permettre la montée en charge horizontale

Le backend est conçu pour rester majoritairement stateless, ce qui favorise la scalabilité.

Dans cette organisation, chaque couche a une responsabilité claire :

- **Couche présentation** : expérience utilisateur (navigation, visualisation, interaction)
- **Couche métier** : point d'entrée unique aux données et traitements (recherche, recommandation, collaboration)
- **Couche données** : concilier analyse relationnelle (Neo4j) et reproductibilité (JSONL + outputs Spark)

Cette séparation facilite aussi le travail en binôme : les développements frontend peuvent progresser sur des contrats d'API stables pendant que la modélisation graphe et les modules IA/Big Data évoluent côté backend.

Cette architecture en couches présente plusieurs avantages structurants :

- **Backend comme façade** : centralise l'authentification (JWT), applique les règles d'accès (RBAC) et expose une API homogène aux modules (graphe, IA, Big Data, collaboration)
- **Couche données hybride** : Neo4j apporte les traversées et requêtes relationnelles, tandis que les JSONL garantissent un mode dégradé pour la démonstration et la reproductibilité
- **Séparation en services** : facilite la conteneurisation (Docker Compose), chaque composant évolue et se déploie indépendamment via des contrats d'API stables
- **Scalabilité** : backend stateless pour montée en charge horizontale, cache pour requêtes fréquentes, WebSocket pour éviter le polling

Le principe API-First structure l'intégration :

- Routes exposées sous `/api/`
- Documentation automatique via OpenAPI
- Communications temps réel (messagerie, workspaces) via WebSocket pour interactions bidirectionnelles

Cette approche facilite l'intégration avec des systèmes tiers :

- Système RH peut interroger l'API pour des statistiques sur les compétences
- Outil de gestion de projet peut récupérer des informations sur les experts disponibles
- Documentation OpenAPI comme contrat d'interface pour les développeurs externes

L'adoption de standards web (REST, WebSocket, JSON) garantit :

- Compatibilité maximale avec les navigateurs modernes et frameworks JavaScript
- Frontend React peut consommer les API via Axios ou Fetch
- Communication WebSocket gérée par bibliothèques spécialisées avec reconnexion automatique
- Standardisation réduit la courbe d'apprentissage et facilite l'onboarding technique

Le déploiement via Docker Compose rend l'environnement reproductible :

- Frontend servi via Nginx
- Neo4j isolé en service dédié
- Healthcheck vérifie Neo4j avant le démarrage du backend

Cette approche conteneurisée présente des avantages opérationnels :

- **Élimination des problèmes de configuration** : encapsulation des dépendances dans les images Docker, évite le "ça marche sur ma machine"
- **Isolation stricte** : chaque service dans son propre conteneur avec ses variables d'environnement, évite les conflits de versions
- **Orchestration simplifiée** : Docker Compose définit les dépendances, réseaux, volumes et politiques de redémarrage

Les healthchecks intégrés assurent :

- Services ne démarrent que lorsque leurs dépendances sont prêtes
- Backend attend que Neo4j réponde correctement avant initialisation
- Frontend Nginx attend que le backend soit opérationnel avant de servir l'application
- Évite les erreurs 502/503 lors de l'accès initial
- Robustesse opérationnelle pour démonstrations et présentations

3.2 Structure du Projet

Le backend est organisé par routeurs (**backend/app/routers**) couvrant les domaines fonctionnels :

- Experts, documents, graphe, learning, dashboard, auth
- IA, Big Data, feed, network, messaging, jobs
- Notifications, gamification, skillmap, workspaces, simulateur Kafka

Les modules ML sont isolés sous **backend/app/ml** et les jobs Spark sous **backend/spark**.

Le frontend (**frontend/src**) est organisé en :

- Pages dédiées aux modules : Feed, GraphVisualization, LearningPath, Messaging, TeamBuilder, TeamWorkspace
- Pages d'authentification
- Composants réutilisables
- Routage centralisé dans **App.tsx**

Les données sont fournies au format JSONL dans **data/** :

- Employees, skills, projects, documents
- Catalogues additionnels : companies, technologies

Cette structuration vise à limiter la complexité :

- Chaque sous-système possède ses fichiers, routes et modèles clairement définis
- Backend : découpage en routeurs réduit les conflits, améliore la lisibilité (un domaine fonctionnel = un point de maintenance)
- Frontend : séparation pages/composants évite la duplication, factorise l'UI (layout, navigation, formulaires, tableaux)
- Facilite l'ajout de fonctionnalités sans remettre en cause l'architecture existante

Au niveau du backend :

- Chaque router est un module autonome avec ses propres modèles Pydantic, services métier et dépendances
- Modularité permet de tester chaque router indépendamment

- Facilite la collaboration entre développeurs (travail parallèle sur différents routeurs)
- Modèles partagés extraits dans des modules communs pour éviter la duplication

Concernant le frontend :

- Composants de base (boutons, formulaires, tableaux, cartes) définis dans `components`
- Réutilisabilité garantit uniformité visuelle et réduit le code dupliqué
- Pages encapsulent la logique spécifique tout en s'appuyant sur des composants génériques
- Lazy loading : seuls les composants nécessaires sont chargés lors de la navigation

3.3 Communication entre Couches

Le frontend communique avec le backend via HTTP REST, avec transport de JWT dans l'en-tête `Authorization`. Les pages consomment les endpoints pour alimenter les recherches, dashboards et la visualisation 3D. L'authentification est validée côté backend, puis les dépendances RBAC filtrent l'accès.

Les fonctionnalités temps réel s'appuient sur WebSocket : messagerie (endpoint par utilisateur) et workspaces (endpoint par workspace et utilisateur). Ce canal supporte des signaux d'interface (typing indicator) et améliore l'expérience.

Le backend communique avec Neo4j via Bolt et exécute des requêtes Cypher. Lorsque Neo4j n'est pas disponible, certaines routes utilisent un mécanisme de fallback basé sur les données JSONL.

Le protocole Bolt, utilisé pour la communication entre le backend et Neo4j, est optimisé pour les performances :

- Pipelining des requêtes pour réduire la latence
- Compression des données pour minimiser le transfert réseau
- Pool de connexions automatique géré par le driver Neo4j Python
- Réutilisation des connexions existantes

Les requêtes Cypher sont :

- Paramétrées pour éviter les injections SQL
- Optimisées grâce à l'utilisation d'index sur les propriétés fréquemment recherchées (email, nom, compétences)

Le mécanisme de fallback JSONL est conçu pour être transparent pour l'utilisateur :

- Lorsqu'une requête Neo4j échoue, le backend tente automatiquement de répondre en utilisant les fichiers JSONL stockés localement
- Particulièrement utile pour les démonstrations en environnement contraint (connexion internet limitée, base de données non disponible)

- Permet de valider la logique métier indépendamment de l'infrastructure de base de données
- Note : ce mode dégradé ne supporte pas toutes les fonctionnalités (notamment les traversées de graphe complexes et les calculs de centralité en temps réel)

Pour préserver un comportement homogène :

- Réponses API normalisées (codes HTTP, schémas Pydantic)
- Documentation OpenAPI comme référence unique pour l'intégration
- Appels sensibles protégés par des dépendances d'autorisation
- Échanges temps réel gérés par un Connection Manager
- Conception limite les effets de bord : erreur Neo4j ou timeout externe produit une réponse contrôlée plutôt qu'un échec global

La normalisation des réponses API garantit :

- Frontend peut traiter les réponses de manière prévisible
- Codes HTTP utilisés de manière sémantique :
 - 200 pour les succès
 - 201 pour les créations
 - 400 pour les erreurs de validation
 - 401 pour l'authentification manquante
 - 403 pour l'autorisation refusée
 - 404 pour les ressources introuvables
 - 500 pour les erreurs serveur inattendues
- Schémas Pydantic définissent la structure des données en entrée et en sortie
- Validation automatique et génération de documentation
- Réduction des erreurs d'intégration et facilitation du débogage

Le Connection Manager pour WebSocket joue un rôle crucial :

- Maintient une registry des connexions actives
- Associe chaque connexion à un utilisateur et/ou un workspace
- Gère les événements de connexion et de déconnexion
- Détecte la déconnexion et nettoie les ressources associées (évite les fuites de mémoire)
- Gère la diffusion des messages : identifie tous les utilisateurs connectés à un workspace et leur envoie le message

3.4 Patterns de Conception Utilisés

Les patterns de conception utilisés dans NEXORA sont :

- **Singleton** : pour le driver Neo4j et les caches de modèles ML
- **Dependency Injection** : via **Depends** (FastAPI) pour composer l'authentification et l'autorisation
- **Connection Manager** : pour les composants WebSocket
- **Strategy** : pour les modules ML (recommandation, classification, embeddings, PageRank, GraphRAG)
- **Factory** : pour la génération de données paramétrable (small/medium/large)

Ces patterns contribuent à la testabilité et à la maintenabilité :

- Dépendances (auth, base de données, paramètres) peuvent être substituées ou simulées lors des tests
- Stratégies ML peuvent évoluer indépendamment du reste de l'API
- Ajout d'un nouvel algorithme (ou variante) sans modifier les routes consommatrices
- Remplacement d'une implémentation derrière une interface ou un service, avec conservation des mêmes contrats d'entrée/sortie

Le pattern Singleton :

- Appliqué au driver Neo4j et aux caches de modèles ML
- Garantit qu'il n'existe qu'une seule instance de ces ressources critiques
- Évite la surcharge mémoire liée à la création de connexions multiples
- Assure la cohérence des données (cache de modèles ML reste synchronisé)
- Driver Neo4j Singleton gère automatiquement le pool de connexions
- Améliore significativement les performances des requêtes Cypher

La Dependency Injection via **Depends** :

- Élément central de l'architecture FastAPI
- Chaque endpoint déclare ses dépendances (authentification, base de données, services métier)
- FastAPI résout automatiquement les dépendances avant l'exécution
- Permet de décorer les endpoints avec des gardes d'autorisation sans modifier la logique métier
- Facilite les tests : dépendances remplacées par des mocks ou des stubs

Le pattern Strategy :

- Visible dans les modules ML
- Chaque algorithme de recommandation, classification ou calcul de centralité est une stratégie interchangeable
- Système de recommandation peut utiliser différentes approches (filtrage collaboratif, PageRank personnalisé, similarité cosinus)

- Consommateur (endpoint API) n'a pas à connaître les détails de l'implémentation
- Flexibilité pour expérimenter rapidement de nouveaux algorithmes
- Comparaison des performances sans impacter l'interface utilisateur

3.5 Flux de Données Complet

Le flux global de données comprend :

- **Génération** : fichiers JSONL via un script paramétrable (seed fixe)
- **Ingestion** : dans Neo4j via des scripts Cypher
- **Traitement ML** : modules ML produisent des recommandations et analyses
- **Batch Spark** : jobs Spark calculent des agrégations batch
- **Consommation** : résultats stockés dans `backend/spark/output` et consommés par les endpoints Big Data

Ce flux est conçu pour être **traçable** :

- Relancer la génération avec le même seed pour reproduire les données
- Vérifier que l'ingestion et les traitements produisent des résultats cohérents
- Important pour un projet académique
- Utile en contexte industriel (rejeu d'un pipeline, vérification post-correction, validation de performances)
- Facilite la mise au point de la visualisation 3D et des modules de recommandation

La génération de données JSONL :

- Utilise un algorithme pseudo-aléatoire avec une seed fixe
- Garantit que chaque exécution produit exactement le même jeu de données
- Contraintes de cohérence :
 - Employé associé à des compétences pertinentes pour son département
 - Projet implique des employés avec les compétences requises
 - Documents liés aux projets correspondants
- Cohérence sémantique essentielle pour des résultats pertinents et une visualisation interprétable

L'ingestion dans Neo4j :

- Via des scripts Cypher qui créent les nœuds (Employee, Skill, Project, Document, Company, Technology)
- Créent les relations (HAS_SKILL, WORKS_ON, CONTRIBUTED_TO, etc.)
- Scripts idempotents : peuvent être réexécutés sans créer de doublons
- Facilite les mises à jour incrémentales

- Contraintes d'unicité définies pour garantir l'intégrité des données (ex : un employé ne peut avoir qu'un seul email)

Les modules ML exploitent ces données pour :

- Calculer des métriques et des recommandations
- PageRank : traverse le graphe pour identifier les experts les plus centraux
- Similarité TF-IDF : analyse les descriptions de compétences pour trouver des profils similaires
- Calculs effectués à la demande (mode interactif) ou en batch via Spark

Les jobs Spark traitent les données JSONL pour :

- Produire des agrégations batch :
 - Statistiques sur la distribution des compétences
 - Scores de popularité des documents
 - Classements des experts par département
- Résultats stockés dans des fichiers JSON dans `backend/spark/output`
- Consommés par les endpoints Big Data
- Approche batch permet de traiter de gros volumes efficacement
- Réduit la charge sur Neo4j lors des requêtes fréquentes

Les cas d'usage principaux de NEXORA sont :

- **Authentification** : inscription, connexion, gestion des sessions JWT
- **Recherche d'experts** : recherche sémantique, filtrage par compétences, classement PageRank
- **Recherche de documents** : recherche textuelle, classification par thématique, recommandations
- **Recommandations** : suggestions de compétences, d'experts, de formations
- **Exploration 3D** : navigation interactive dans le graphe organisationnel
- **Assistant conversationnel** : chatbot Veda pour interrogation en langage naturel
- **Collaboration** : messagerie temps réel, workspaces d'équipe, constitution d'équipes

Les entités principales du graphe sont :

- **Expert (Employee)** : employé avec compétences, département, rôle, localisation
- **Skill** : compétence avec catégorie, niveau de demande, description
- **Project** : projet avec statut, date, stack technologique, compétences requises
- **Document** : document avec type, topic, rating, vues, auteur
- **Company** : entreprise avec secteur, localisation, taille
- **Technology** : technologie avec catégorie, popularité

Les relations clés incluent :

- HAS_SKILL : expert possède une compétence (avec niveau)
- WORKS_ON : expert participe à un projet
- CONTRIBUTED_TO : expert a contribué à un document
- REQUIRES_SKILL : projet nécessite une compétence
- RELATED_TO : document lié à un projet ou une compétence

Le pipeline ML de NEXORA comprend les étapes suivantes :

- **Ingestion** : chargement des données JSONL et création des nœuds/reliations Neo4j
- **Normalisation** : nettoyage des textes, tokenisation, lemmatisation
- **Représentation** : calcul des vecteurs TF-IDF pour les textes
- **Classification** : catégorisation automatique des documents par thématique
- **Scoring** : calcul des scores de similarité et de centralité (PageRank)
- **Exposition** : exposition des résultats via l'API REST

Les mécanismes de fallback et de robustesse incluent :

- Fallback JSONL si Neo4j indisponible
- Cache en mémoire pour les calculs fréquents
- Fallback rule-based si LLM (Groq) indisponible
- Batch Spark pour les agrégations lourdes

Le simulateur Kafka illustre un mode streaming par production d'événements. L'ensemble des résultats est exposé via l'API et rendu dans le frontend : dashboards, recherche, visualisation 3D, chat Veda, et modules collaboratifs.

3.6 Gestion des Erreurs

La gestion des erreurs repose sur des exceptions FastAPI traduites en codes HTTP adaptés (401, 403, 404, 422, 500). Les validations Pydantic renforcent la robustesse des entrées. En sécurité, un mécanisme de rate limiting est appliqué au login.

Un mécanisme de fallback JSONL est présent lorsque Neo4j est indisponible, améliorant la continuité de démonstration. Cependant, il impose une gouvernance stricte pour éviter des divergences.

L'objectif principal est de garantir une **dégradation contrôlée**. Lorsque la dépendance critique (base graphe ou service externe) n'est pas accessible, l'API privilégie une réponse partielle ou alternative plutôt qu'une rupture. Cette approche améliore l'expérience utilisateur (les pages restent fonctionnelles) et facilite la démonstration : on peut montrer la logique métier même en cas d'indisponibilité temporaire d'un composant.

Chapitre 4

Gestion de Projet

4.1 Objectifs et Périmètre

Cette section présente la démarche de **gestion de projet** appliquée à NEXORA. L'objectif est de montrer comment le travail a été cadré, découpé, planifié et piloté afin de produire un livrable complet (backend, frontend, base graphe, IA/Big Data, déploiement et documentation). Le périmètre couvre :

- la définition des objectifs et des contraintes (sécurité, performance, robustesse),
- le découpage en lots et la planification par dépendances,
- la répartition des responsabilités au sein du binôme,
- l'anticipation des risques et la mise en place de mitigations.

4.2 Charte de Projet (Cadrage)

Le cadrage de NEXORA a consisté à formaliser le **besoin**, les **objectifs**, les **parties prenantes** et les **critères d'acceptation**. Le problème adressé est la fragmentation de la connaissance des compétences dans les organisations, ce qui complexifie l'identification d'experts, la collaboration et la capitalisation documentaire.

Les objectifs ont été définis autour d'un socle fonctionnel (recherche d'experts et documents, visualisation graphe), d'un socle d'intelligence (recommandations, classification, ranking), et d'un socle d'industrialisation (sécurité, déploiement reproductible, tests, documentation).

4.3 Cahier des Charges (Expression des Besoins)

Les besoins de NEXORA ont été exprimés selon une logique **fonctionnelle** et **non fonctionnelle**, afin d'assurer une cohérence entre le produit attendu et l'implémentation.

Le cahier des charges de NEXORA peut être synthétisé autour de :

- **Fonctionnel** : recherche experts/documents, visualisation graphe, recommandations, collaboration (messaging/workspaces),
- **Non fonctionnel** : sécurité JWT+RBAC, robustesse (fallback), performance (cache ML, sous-graphes),
- **Livrables** : application conteneurisée, documentation API (Swagger), rapport et démonstration.

4.4 Découpage (WBS) et Lots

Pour NEXORA, le découpage en sous-problèmes a été déterminant pour maîtriser la complexité (graphe + IA + UI + DevOps). Le WBS a été construit pour obtenir des incréments démontrables, en limitant les intégrations tardives et en rendant explicites les dépendances entre modules.

Ce découpage favorise l'alignement entre architecture technique et gouvernance du projet : chaque lot produit un incrément démontrable et testable.

TABLE 4.1 – WBS (Work Breakdown Structure) spécifique à NEXORA

Lot	Sous-lots	Livrables / critères de sortie
Lot 1	Données & Graphe	Génération JSONL (échelles), modélisation Neo4j, requêtes Cypher, endpoints graphe pour visualisation 3D
Lot 2	Backend API	FastAPI (routeurs), sécurité JWT+RBAC, endpoints IA (TF-IDF, classification, PageRank), endpoints Big Data (Spark outputs + fallback), WebSocket (messaging/workspaces)
Lot 3	Frontend	Pages React (routing), ProtectedRoute, TopNavBar, intégration API, visualisation 3D, chatbot Veda, écrans de recherche/recommandation
Lot 4	Qualité & Déploiement	Docker Compose, healthchecks, tests backend (Pytest), Swagger opérationnel, stabilité des erreurs (401/403/422/500), rapport final

4.5 Planification : Diagramme de Gantt et Chemin Critique

4.5.1 Diagramme de Gantt

Le diagramme de Gantt ci-dessous illustre l'ordonnancement des principales tâches du projet NEXORA sur 8 semaines. Les barres rouges identifient le **chemin critique** (marge nulle) et les barres bleues les tâches disposant d'une marge. Les flèches matérialisent les dépendances.

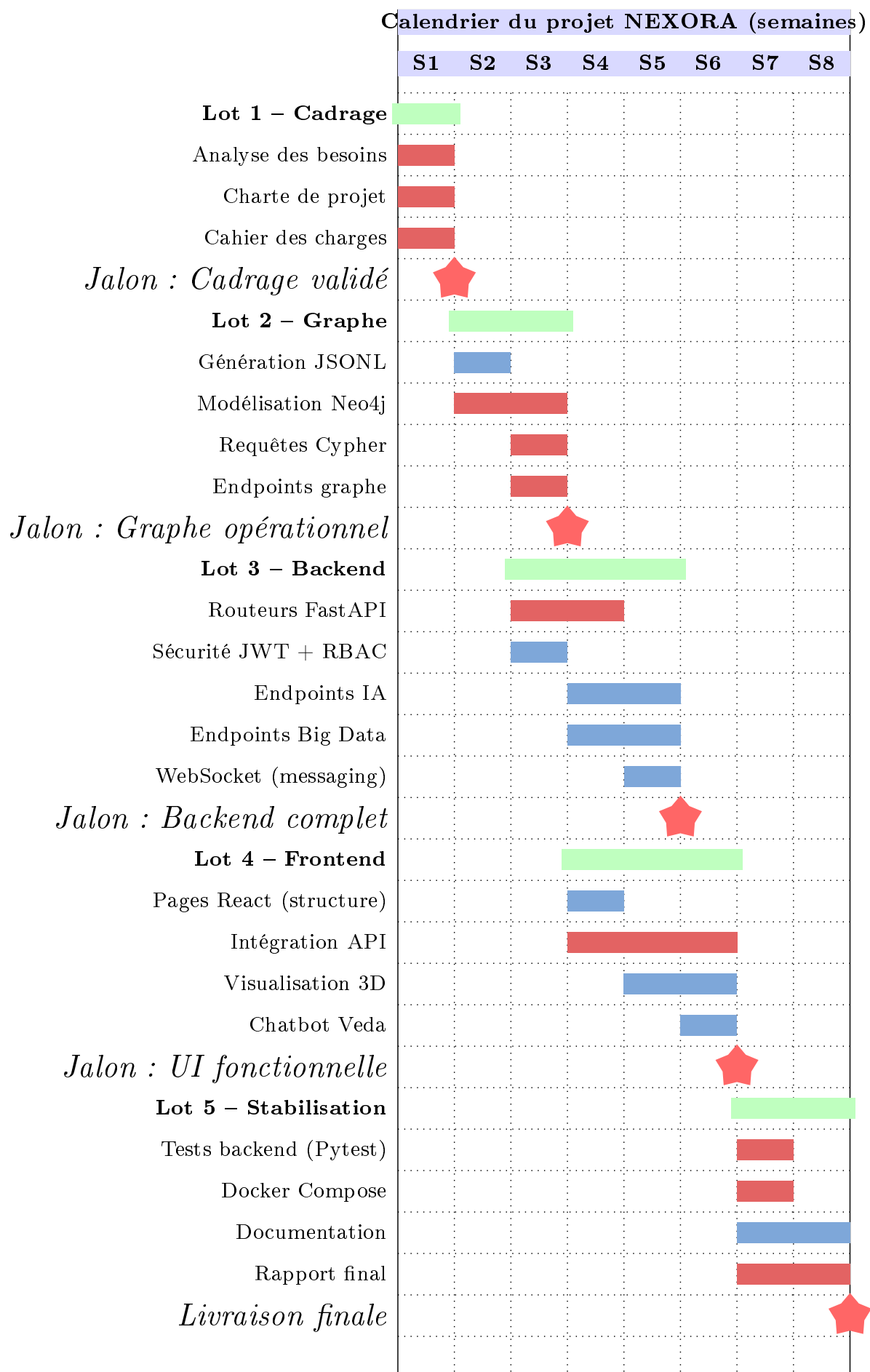


FIGURE 4.1 – Diagramme de Gantt de NEXORA avec chemin critique (rouge) et dépendances

4.5.2 Chemin Critique (méthode CPM)

L'identification du **chemin critique** selon la méthode CPM (*Critical Path Method*) repère les tâches dont la marge totale est nulle. Tout retard sur l'une de ces tâches décale la date de livraison. Le chemin critique de NEXORA est :

**Analyse → CDC → Mod. Neo4j → Cypher → Endp. graphe → Routeurs →
Intégration API → Tests → Docker → Rapport → Livraison**

La durée totale selon le chemin critique est de **8 semaines**. Les 10 tâches non critiques (Génération JSONL, Sécurité JWT, Endpoints IA/Big Data, WebSocket, Pages React, Visualisation 3D, Chatbot Veda, Documentation) disposent chacune d'une marge d'une semaine.

4.5.3 Tableau PERT : tâches, durées, dépendances et marges

TABLE 4.2 – Analyse PERT — calcul des dates et marges (en semaines)

Tâche	Dur.	Dépendances	ES	EF	LS	LF	Marge	Critique
Analyse des besoins	1	–	S1	S1	S1	S1	0	✓
Charte de projet	1	–	S1	S1	S1	S1	0	✓
Cahier des charges	1	Analyse	S1	S1	S1	S1	0	✓
Génération JSONL	1	CDC	S2	S2	S3	S3	1	
Modélisation Neo4j	2	Gén. JSONL	S2	S3	S2	S3	0	✓
Requêtes Cypher	1	Mod. Neo4j	S3	S3	S3	S3	0	✓
Endpoints graphe	1	Req. Cypher	S3	S3	S3	S3	0	✓
Routeurs FastAPI	2	Endp. graphe	S3	S4	S3	S4	0	✓
Sécurité JWT + RBAC	1	Routeurs	S3	S3	S4	S4	1	
Endpoints IA	2	Sécurité JWT	S4	S5	S5	S6	1	
Endpoints Big Data	2	Routeurs	S4	S5	S5	S6	1	
WebSocket (messaging)	1	Routeurs	S5	S5	S6	S6	1	
Pages React	1	Endp. graphe	S4	S4	S5	S5	1	
Intégration API	3	Pages React	S4	S6	S4	S6	0	✓
Visualisation 3D	2	Intégr. API	S5	S6	S6	S7	1	
Chatbot Veda	1	Endpoints IA	S6	S6	S7	S7	1	
Tests backend	1	Intégr. API	S7	S7	S7	S7	0	✓
Docker Compose	1	Tests backend	S7	S7	S7	S7	0	✓
Documentation	2	Tests backend	S7	S8	S8	S8	1	
Rapport final	2	Toutes tâches	S7	S8	S7	S8	0	✓

Légende : ES = Early Start, EF = Early Finish, LS = Late Start, LF = Late Finish. La marge totale est calculée comme $MT = LS - ES$; les tâches à marge nulle (✓) constituent le chemin critique.

4.6 Méthodologie Itérative et Backlog Produit

Le développement de NEXORA a suivi une démarche **itérative** organisée en **4 sprints** de 2 semaines chacun. Chaque sprint produit un incrément démontrable et testable, réduisant le risque d'intégration tardive.

TABLE 4.3 – Backlog produit organisé par sprints (priorité MoSCoW)

Sprint	User Stories / Fonctionnalités	Priorité	Critères d'acceptation
Sprint 1 (S1–S2)	Cadrage : charte, CDC, analyse besoins	Must	Documents validés par l'encadrante
	Génération JSONL + modèle Neo4j	Must	Données chargées, requêtes fonctionnelles
	Schéma graphe + requêtes Cypher	Must	Requêtes testées en console Neo4j
Sprint 2 (S3–S4)	Routeurs FastAPI + sécurité JWT/RBAC	Must	Auth fonctionnelle, gardes testées
	Endpoints graphe + IA (TF-IDF, PageRank)	Must	Swagger documenté, réponses 200
	Pages React + routing protégé	Should	Navigation fonctionnelle, ProtectedRoute
Sprint 3 (S5–S6)	Intégration API frontend + visualisation 3D	Must	Graphe 3D navigable, filtres actifs
	Chatbot Veda (LLM + fallback)	Should	Réponses pertinentes, mode dégradé
	WebSocket (messaging, workspaces)	Could	Messages temps réel fonctionnels
Sprint 4 (S7–S8)	Tests backend (Pytest)	Must	Tests passants, couverture >70%
	Docker Compose + health-checks	Must	Déploiement reproductible
	Documentation + rapport final	Must	Rapport compilable, Swagger à jour

4.6.1 Suivi d'avancement par sprint

TABLE 4.4 – Tableau de bord d'avancement par sprint

Sprint	Stories planifiées	Stories livrées	Taux	Statut
Sprint 1	3	3	100%	✓ Terminé
Sprint 2	3	3	100%	✓ Terminé
Sprint 3	3	3	100%	✓ Terminé
Sprint 4	3	3	100%	✓ Terminé
Total	12	12	100%	Livré

4.7 Matrice RACI

La matrice RACI ci-dessous formalise les responsabilités selon quatre rôles : **R**esponsable (exécute), **A**pprobateur (valide), **C**onsulté (fournit un avis), **I**nformé (tenu au courant).

TABLE 4.5 – Matrice RACI détaillée pour NEXORA

Activité	Soubaibi Omar	Yassine Baizi	Mme Ounacer
Cadrage (charte, CDC)	R/A	R	C
Architecture globale	R/A	C	I
Modélisation Neo4j + requêtes Cypher	R/A	C	I
Backend FastAPI (routeurs, sécurité)	R/A	C	I
Modules IA (TF-IDF, Page-Rank, Veda)	R/A	C	I
Endpoints Big Data (Spark)	R	R/A	I
Frontend React (pages, composants)	C	R/A	I
Intégration API frontend	C	R/A	I
Visualisation 3D WebGL	C	R/A	I
Déploiement Docker Compose	R/A	C	I
Tests backend (Pytest)	R	R	I
Rédaction du rapport	R/A	R	C
Validation des livrables	I	I	A

4.8 Matrice des Risques

TABLE 4.6 – Matrice des risques et plans de mitigation (NEXORA)

Risque	Prob.	Impact	Mitigation
Indisponibilité Neo4j	M	Élevé	Fallback JSONL, gestion d'erreur centralisée, health-check Docker
Indisponibilité Groq / clé absente	Élevé	Moyen	Fallback rule-based (Veda), gestion d'erreur + réponse dégradée
Dégradation performance du graphe 3D	M	Moyen	Filtrage/sous-graphes, limitation densité, optimisation rendu
Latences ML (TF-IDF/classification)	M	Moyen	Cache en mémoire, initialisation paresseuse, réutilisation modèles
Régressions sécurité (RBAC)	Faible	Élevé	Gardes centralisées, tests auth, contrôles 401/403
Incohérence données (JSONL ↔ Neo4j)	M	Moyen	Générateur reproductible, schéma stable, validations Pydantic

4.9 Indicateurs de Suivi (KPI)

Les indicateurs ci-dessous offrent une vue synthétique de l'avancement et de la qualité du projet.

TABLE 4.7 – Tableau de bord des KPI du projet NEXORA

Indicateur	Valeur	Observation
Durée totale du projet	8 semaines	Conforme au planning
Nombre de sprints	4	Sprints de 2 semaines
Stories livrées / planifiées	12/12 (100%)	Aucun report
Lignes de code	17 200+	Backend + Frontend + ML
Fichiers source	~100	Organisation modulaire
Pages frontend	17	Toutes fonctionnelles
Routeurs backend	19	Documentés via Swagger
Modules ML	9	Testés unitairement
Couverture tests (backend)	>70%	Pytest + TestClient
Endpoints documentés	100%	OpenAPI/Swagger
Taux de disponibilité (démon)	>99%	Fallback JSONL actif

4.10 Synthèse de Gestion de Projet

Le tableau ci-dessous synthétise les réponses aux questions classiques de gestion de projet, directement ancrées dans les choix de NEXORA.

TABLE 4.8 – Synthèse des pratiques de gestion de projet appliquées

Question GP	Réponse dans NEXORA
Découpage du projet	WBS en 5 lots (Tableau 4.1) : Cadrage, Graphe, Backend, Frontend, Stabilisation. Chaque lot produit un incrément démontrable.
Dépendances	Le schéma Neo4j conditionne les endpoints, qui conditionnent l'intégration frontend (voir liens du Gantt, Figure 4.1).
Chemin critique	11 tâches à marge nulle (Tableau 4.2), durée totale : 8 semaines.
Durée du projet	8 semaines (4 sprints de 2 semaines), conforme au planning macro.
Ressources	Binôme avec répartition claire (RACI, Tableau 4.5) : backend/IA vs frontend/UX.
Risques	6 risques identifiés avec mitigations implémentées (Tableau 4.6).
Qualité	Pydantic (validation), Swagger (documentation), Pytest (tests), Docker (reproductibilité).
Méthodologie	Itérative en 4 sprints avec backlog MoSCoW (Tableau 4.3).

Chapitre 5

Études Stratégiques et de Faisabilité

5.1 Analyse PESTEL

TABLE 5.1 – Analyse PESTEL de NEXORA

Facteur	Influence sur le projet	Évaluation
Politique	Stratégies nationales de transformation digitale	Positive : soutien étatique au e-gouvernement et RH numériques
Économique	Optimisation des coûts RH recherchée par les entreprises	Positive : ROI attractif pour cartographie des compétences
Socioculturel	Acceptation des outils IA en gestion RH	Mitigée : freins culturels possibles, nécessité d'accompagnement
Technologique	Maturité des graphes Neo4j et LLM (Groq)	Positive : technologies accessibles et matures
Environnemental	Réduction des déplacements grâce au télétravail	Positive : favorise la gestion des compétences à distance
Légal	RGPD, protection des données RH	Contrainte : nécessite conformité stricte

5.2 Analyse SWOT

TABLE 5.2 – Analyse SWOT de NEXORA

Forces (Internes)	
Graphe de connaissances natif (Neo4j)	Visualisation 3D interactive (WebGL)
Architecture API-First modulaire	Approche IA hybride (ML + LLM avec fallback)
Déploiement reproductible (Docker Compose)	Sécurité JWT + RBAC
Faiblesses (Internes)	
Streaming simulé (Kafka simulé)	Dépendance à Groq pour LLM avancé
Fallback JSONL nécessitant gouvernance	Courbe d'apprentissage pour Neo4j/Cypher
Opportunités (Externes)	
Marché croissant de la Talent Intelligence	Pénurie de solutions open source en cartographie RH
Adoption accélérée du télétravail	Potentiel d'intégration avec Moodle/SAP
Menaces (Externes)	
Solutions propriétaires dominantes (Workday, 365Talents)	Évolutions rapides des LLM
Exigences RGPD et conformité	Résistance culturelle aux outils IA

5.3 Analyse des 5 Forces de Porter

TABLE 5.3 – Analyse des 5 forces de Porter pour NEXORA

Force	Influence	Justification
Intensité de la concurrence	Élevée	Présence de Workday, 365Talents, Cornerstone ; mais absence de solution open source équivalente
Pouvoir des clients	Moyen	Les entreprises ont des budgets RH limités mais recherchent des solutions visibles rapidement
Pouvoir des fournisseurs	Faible	Technologies open source (Neo4j, FastAPI, React) ; seul Groq est propriétaire mais avec fallback
Menace des nouveaux entrants	Moyenne	Barrière technique (graphe + IA + Big Data) mais possible avec investissement
Menace des substituts	Élevée	Solutions Excel/SharePoint, réseaux informels, consultance externe

5.4 Analyse Croisée et Recommandations Stratégiques

Le croisement des analyses PESTEL, SWOT et Porter permet de dégager des **recommandations stratégiques** concrètes pour le positionnement de NEXORA.

TABLE 5.4 – Matrice de recommandations stratégiques (croisement SWOT-PESTEL)

Axe stratégique	Opportunité / Menace	Action NEXORA
SO (Forces $\times$ Opportunités)	Marché Talent Intelligence en croissance + Graphe natif Neo4j	Positionner NEXORA comme alternative open source aux solutions propriétaires
WO (Faiblesses $\times$ Opportunités)	Streaming simulé + Adoption télétravail	Migrer vers Kafka réel pour valoriser l'offre temps réel
ST (Forces $\times$ Menaces)	IA hybride + Solutions propriétaires dominantes	Renforcer l'explicabilité (atout vs boîtes noires) et la conformité RGPD
WT (Faiblesses $\times$ Menaces)	Dépendance Groq + Évolution rapide des LLM	Maintenir le fallback rule-based et rester agnostique vis-à-vis du fournisseur LLM

5.4.1 Business Model Canvas

Le Business Model Canvas ci-dessous synthétise le modèle économique de NEXORA en 9 blocs.

TABLE 5.5 – Business Model Canvas de NEXORA

Bloc	Description
Proposition de valeur	Cartographie intelligente des compétences via graphe Neo4j, recommandation IA explicable, visualisation 3D immersive, chatbot Veda
Segments clients	DSI et DRH de moyennes/grandes entreprises, cabinets de conseil RH, établissements d'enseignement
Canaux	SaaS cloud (AWS/DigitalOcean), déploiement on-premise (Docker), démonstrations en ligne, webinaires
Relation client	Support technique, documentation API ouverte, communauté open source, accompagnement à la mise en place
Sources de revenus	Freemium (<100 employés gratuit), licence annuelle entreprise (50 000 MAD/an), services de personnalisation
Ressources clés	Équipe de développement (fullstack, data engineer), infrastructure cloud, base de connaissances Neo4j
Activités clés	Développement plateforme, maintenance et mises à jour, support client, veille technologique (LLM, graphe)
Partenaires clés	Neo4j (technologie graphe), Groq (LLM), fournisseurs cloud (AWS/DO), EMSI (cadre académique)
Structure de coûts	Hébergement cloud (~500 MAD/mois), ressources humaines, API Groq (freemium), domaine et certificats SSL

5.5 Étude Technique

5.5.1 Besoins Matériels

TABLE 5.6 – Matériels informatiques nécessaires

Nature	Prix Unitaire (MAD)	Quantité	Total (MAD)
Serveur (ou VM) 16 Go RAM, 4 vCPU	8000	1	8000
Poste de développement (8 Go RAM mini)	6000	2	12000
Stockage SSD 256 Go	800	2	1600

5.5.2 Ressources Humaines

TABLE 5.7 – Ressources humaines nécessaires

Poste	Coût mensuel (MAD)	Nombre	Total (MAD)
Chef de projet / Développeur Fullstack	15000	1	15000
Développeur Frontend React	12000	1	12000
Data Engineer (Neo4j, Spark)	14000	1	14000

5.5.3 Logiciels et Services

TABLE 5.8 – Logiciels et services

Ressource	Prix Unitaire (MAD)	Quantité	Total (MAD)
Licence Neo4j (Community gratuite)	0	1	0
Hébergement cloud (AWS/DigitalOcean)	500	4 mois	2000
Nom de domaine (annuel)	120	1	120
API Groq (freemium, crédits)	0	1	0

5.6 Étude Commerciale et Marketing

TABLE 5.9 – Stratégie marketing (4P) pour NEXORA

Politique	Stratégie adoptée
Prix	Modèle Freemium : version gratuite pour PME (max 100 employés); Licence annuelle pour grandes entreprises (à partir de 50 000 MAD/an)
Place	SAAS sans déplacement : déploiement cloud (AWS/DigitalOcean) avec assistance à distance; option sur site pour RGPD
Promotion	Publicité en ligne (LinkedIn, Google Ads), démonstrations gratuites, webinaires techniques, documentation open source
Produit	Plateforme SaaS d'intelligence organisationnelle avec API ouverte, graphe Neo4j, IA, visualisation 3D

5.6.1 Outils et services acquis

- **Outil spécifique** : Aucun outil matériel supplémentaire; utilisation d'ordinateurs de développement standards.
- **Service spécifique** : Nom de domaine (nexora.ai) et hébergement cloud (DigitalOcean) pour démonstration.

5.7 Étude Financière Simplifiée

TABLE 5.10 – Budget de prospection et ventes (semestriel)

Indicateur	S1 (lancement)	S2 (+6 mois)	Total
Budget prospection (MAD)	20 000	15 000	35 000
Nombre de prospects ciblés	500	400	900
Taux de conversion	5%	8%	–
Nombre d'acheteurs	25	32	57
Chiffre d'affaires (MAD) (licence à 50 000 MAD/an)	1 250 000	1 600 000	2 850 000
Taux de churn (abandon)	–	10%	–

- **Coût total du projet (développement 4 mois) :** environ 80 000 MAD (ressources humaines + hébergement + domaine)
- **Retour sur investissement (ROI) estimé :** environ 35x après 1 an
- **Seuil de rentabilité :** environ 12 clients annuels

Chapitre 6

Conception et Implémentation Technique

6.1 Diagrammes UML

Cette section regroupe l'ensemble des diagrammes de conception utilisés dans NEXORA. Ils fournissent une vue structurée de l'architecture, des interactions et du modèle de données du système. Chaque diagramme est accompagné d'une description détaillée expliquant son rôle dans la conception globale.

6.1.1 Architecture 3-tiers

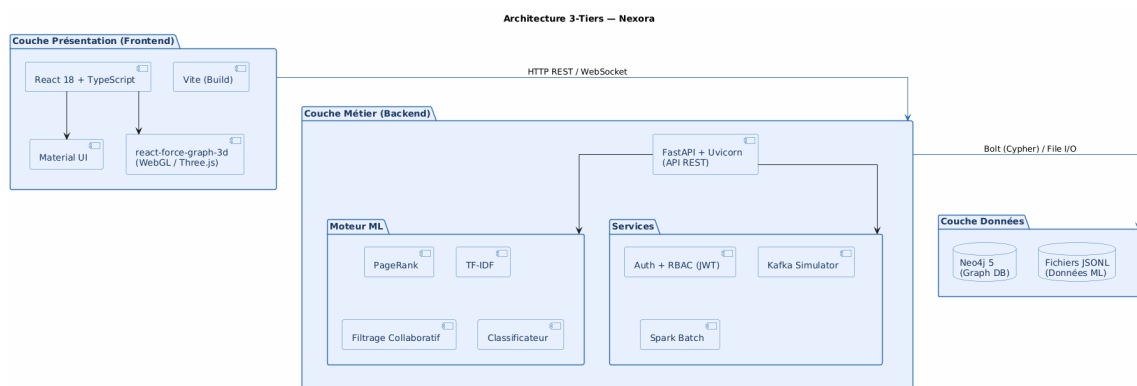


FIGURE 6.1 – Architecture 3-tiers de NEXORA

Le diagramme d'architecture 3-tiers (Figure 6.1) décompose le système NEXORA en trois couches distinctes. La **couche présentation** (React 18 + TypeScript) gère l'interface utilisateur, le routage et les interactions. La **couche métier** (FastAPI) centralise la logique applicative : authentification JWT, modules IA, endpoints REST et WebSocket. La **couche données** combine Neo4j (graphe), les fichiers JSONL (fallback) et les sor-

ties Spark (batch). Les flèches indiquent les flux de communication : HTTP/REST entre frontend et backend, protocole Bolt entre backend et Neo4j, et lecture de fichiers pour les données JSONL et Spark.

6.1.2 Diagramme de cas d'usage

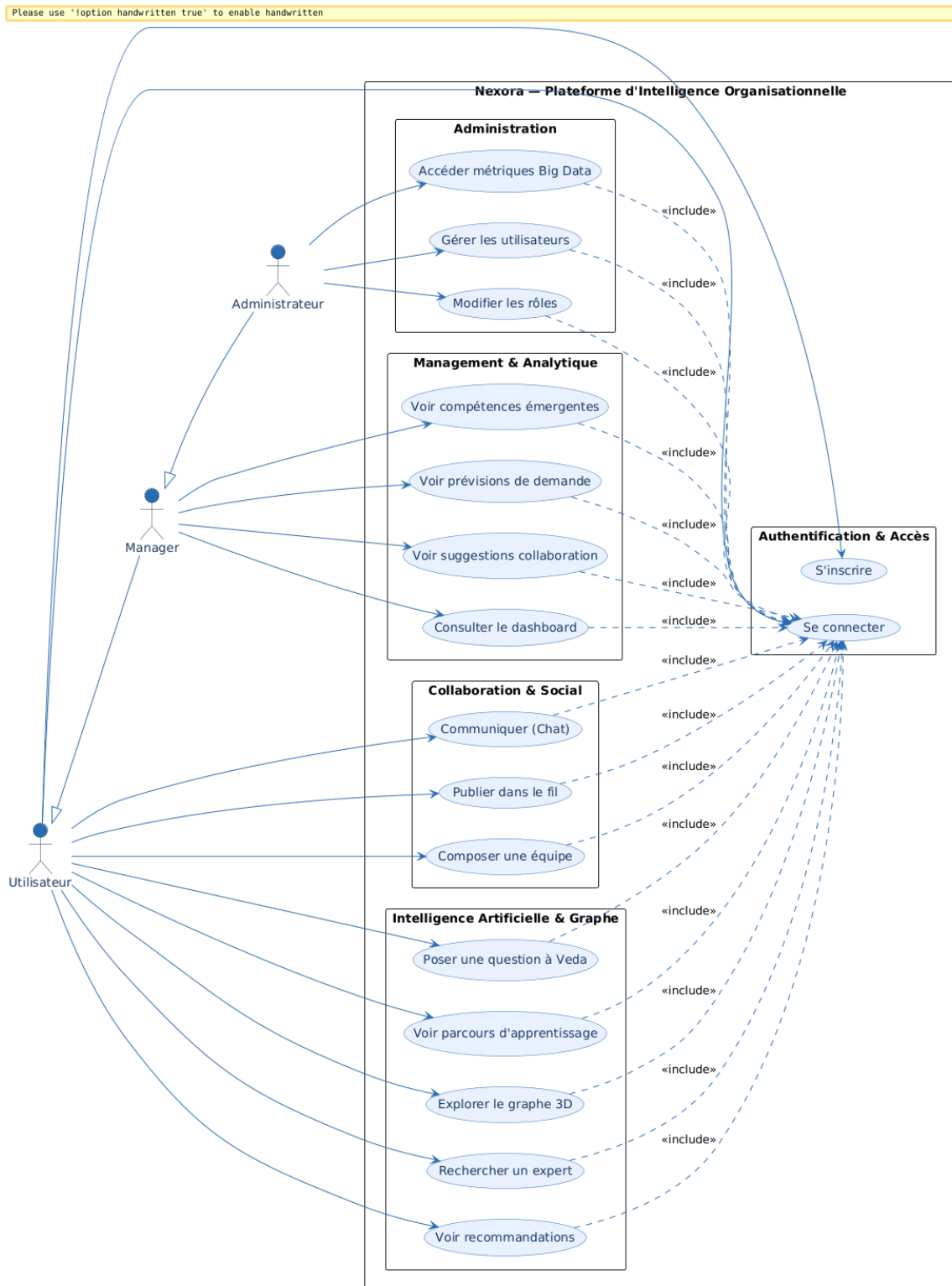
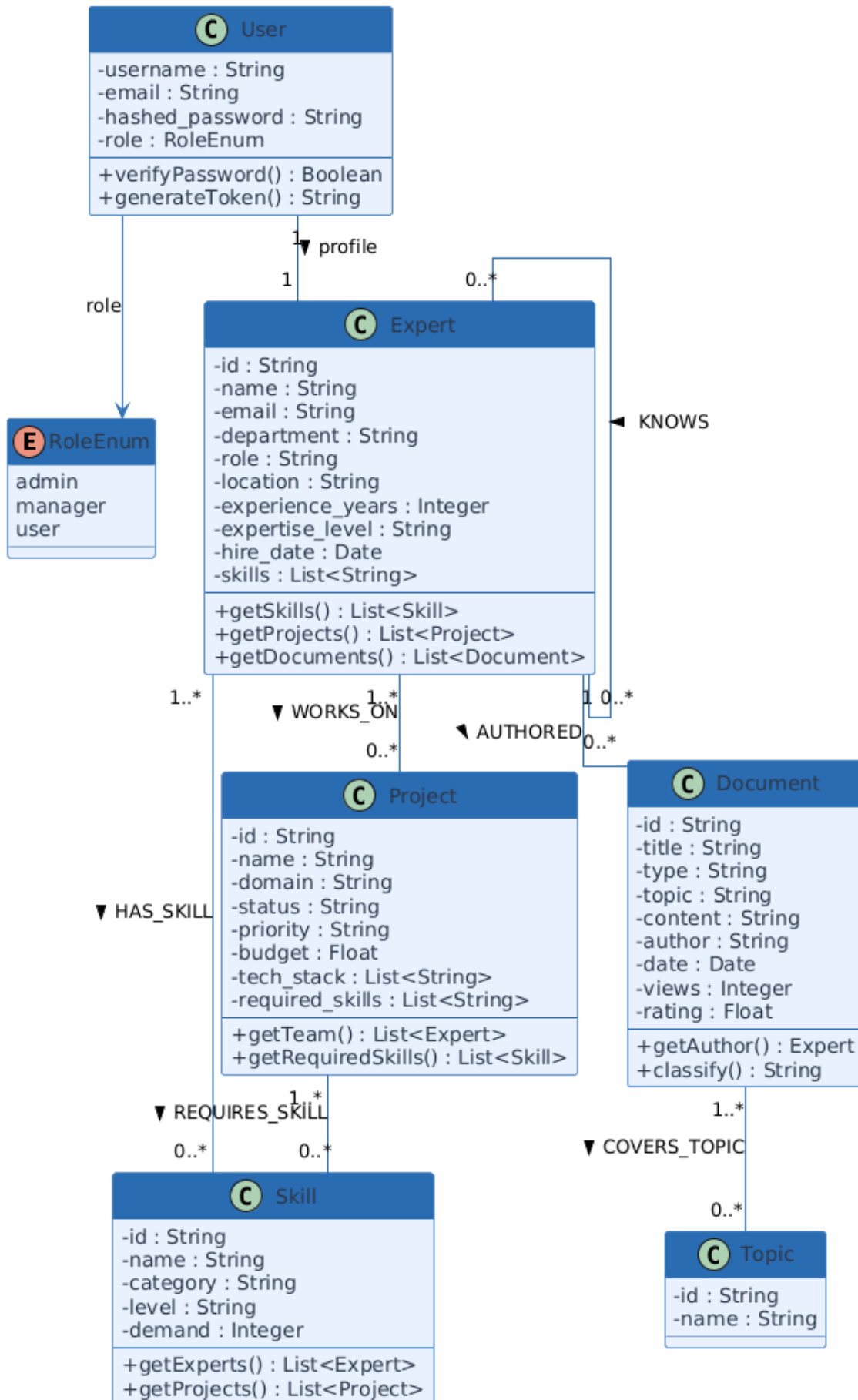


FIGURE 6.2 – Cas d'usage principaux de NEXORA

Le diagramme de cas d'usage (Figure 6.2) identifie les acteurs du système et leurs interactions. Trois acteurs principaux sont représentés : l'**Utilisateur** (recherche d'experts,

exploration du graphe, messagerie), le **Manager** (constitution d'équipes, accès au Team Builder) et l'**Administrateur** (gestion des utilisateurs, configuration RBAC). Les cas d'usage couvrent l'authentification, la recherche sémantique, la recommandation d'experts, la visualisation 3D, l'interaction avec le chatbot Veda, la messagerie temps réel et l'administration. Les relations d'inclusion (*include*) montrent les dépendances : par exemple, toute action nécessite une authentification préalable.

6.1.3 Diagramme de classes



Le diagramme de classes (Figure 6.3) modélise la structure statique du backend. Les classes principales incluent : **User** (attributs d'authentification et rôle RBAC), **Expert** (compétences, département, expérience), **Skill** (catégorie, niveau de demande), **Project** (statut, stack technique) et **Document** (type, topic, auteur). Les associations reflètent les relations Neo4j : un Expert *possède* des Skills (HAS_SKILL), *participe* à des Projects (WORKS_ON) et *contribue* à des Documents (CONTRIBUTED_TO). Les classes de service (**AuthService**, **RecommendationService**, **GraphService**) encapsulent la logique métier et sont injectées via le pattern Dependency Injection de FastAPI.

6.1.4 Diagramme de séquence — Authentification

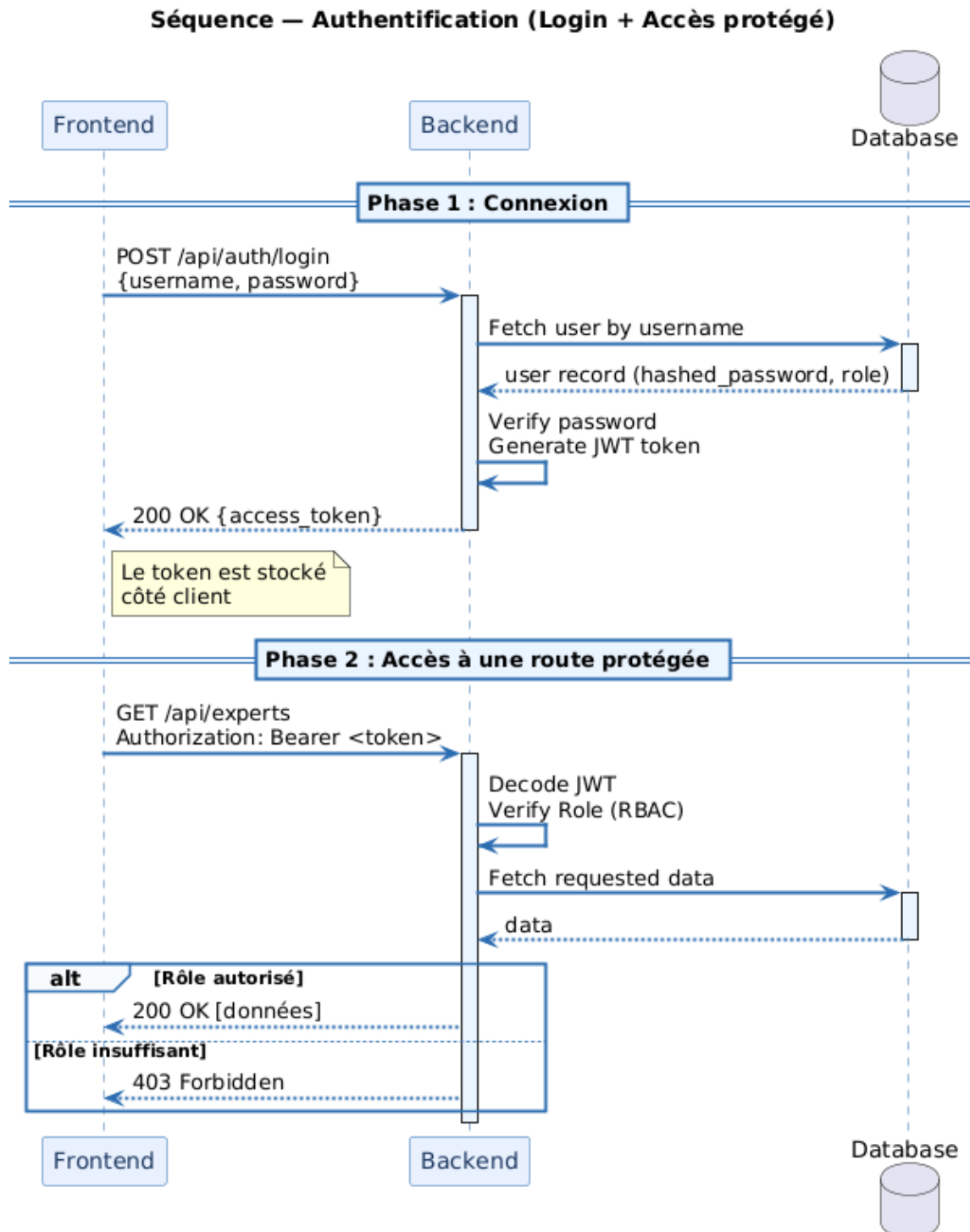


FIGURE 6.4 – Séquence d'authentification : login, génération et validation JWT

Le diagramme de séquence (Figure 6.4) détaille le flux d'authentification en six étapes. (1) L'utilisateur soumet ses identifiants via le formulaire de connexion. (2) Le frontend envoie une requête POST à `/api/auth/login`. (3) Le backend vérifie les identifiants contre la base Neo4j et applique le rate limiting. (4) Si les identifiants sont valides, un token JWT

est généré avec les claims **sub**, **role** et **exp**. (5) Le token est retourné au frontend qui le stocke dans le localStorage. (6) Les requêtes ultérieures incluent le token dans l'en-tête **Authorization: Bearer**, permettant au backend de valider l'identité et les droits sans maintenir de session.

6.1.5 Schéma de déploiement

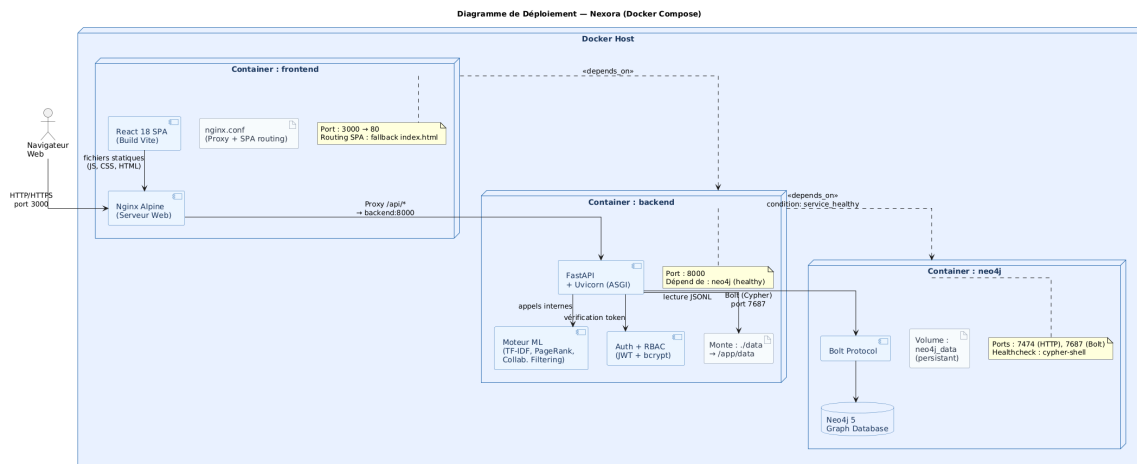


FIGURE 6.5 – Schéma de déploiement (Docker Compose / services)

Le schéma de déploiement (Figure 6.5) représente l'infrastructure conteneurisée de NEXORA. Trois services Docker sont orchestrés via Docker Compose : le conteneur **Neo4j** (ports 7474/7687, volume persistant pour les données), le conteneur **Backend** (FastAPI sur le port 8000, dépendant du healthcheck Neo4j) et le conteneur **Frontend** (Nginx servant l'application React sur le port 3000). Les healthchecks assurent un démarrage ordonné : Neo4j doit répondre avant le backend, et le backend avant le frontend. Le réseau Docker interne isole les communications inter-services.

6.1.6 Schéma du graphe de données

Schéma du Graphe — Modèle de Données Neo4j

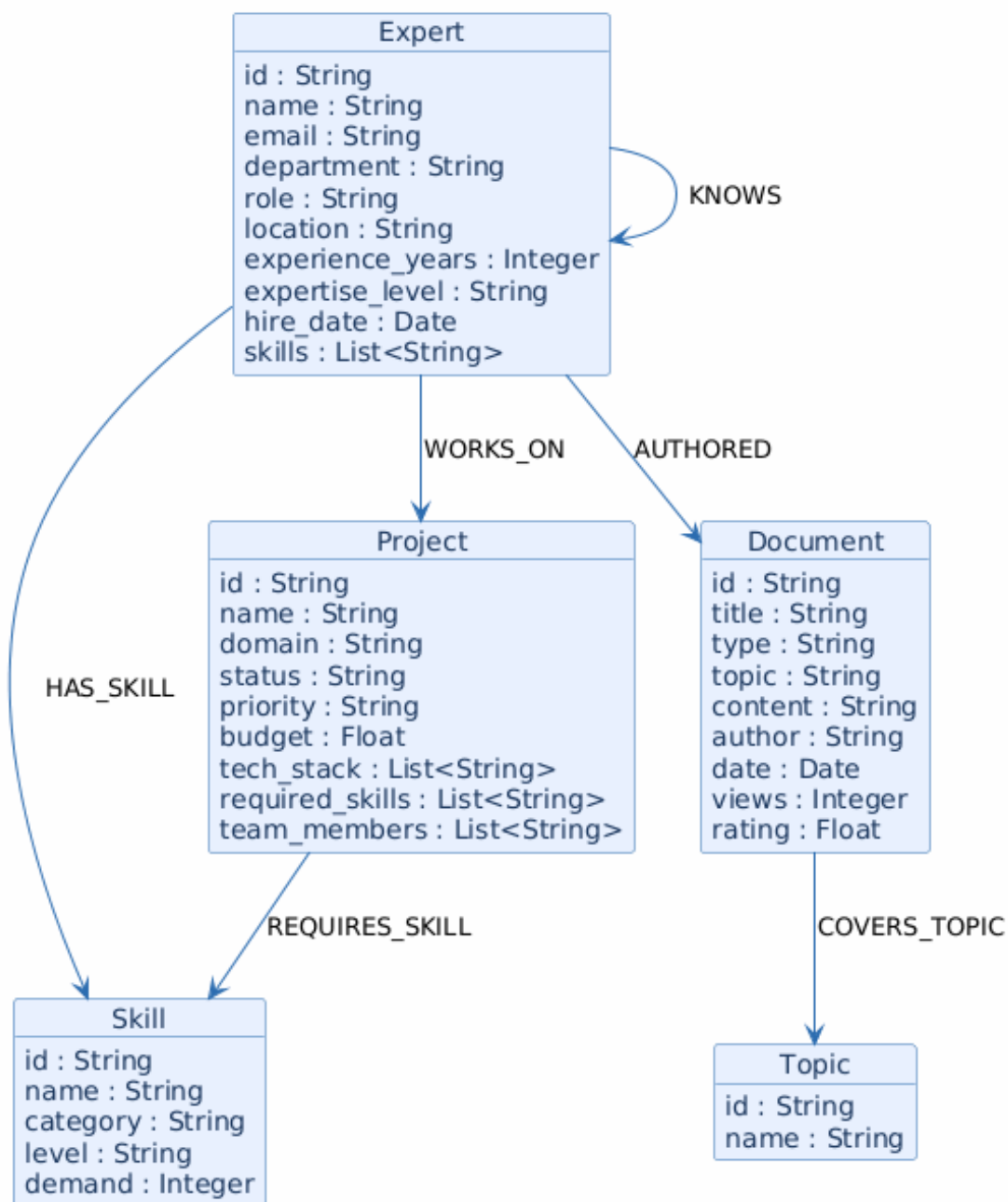


FIGURE 6.6 – Schéma et relations clés du graphe (représentation)

Le schéma du graphe (Figure 6.6) représente le modèle de données Neo4j de NEXORA. Les nœuds principaux sont : **Expert** (employé avec ses attributs), **Skill** (compétence normalisée), **Project** (projet avec statut et stack) et **Document** (production documentaire). Les relations typées (**HAS_SKILL**, **WORKS_ON**, **CONTRIBUTED_TO**, **REQUIRES_SKILL**, **RELATED_TO**) portent des propriétés (niveau de maîtrise, date, rôle). Ce schéma permet les traversées de graphe nécessaires aux algorithmes de cen-

tralité (PageRank), de recommandation (similarité cosinus) et d'explicabilité (chemins justificatifs).

6.1.7 Modèle de données

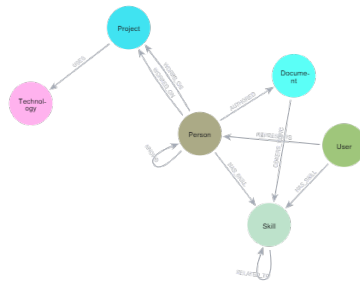


FIGURE 6.7 – Modèle de données (vue applicative et entités principales)

Le modèle de données applicatif (Figure 6.7) illustre le lien entre les fichiers JSONL (source), le graphe Neo4j (stockage) et l'API (exposition). Chaque entité possède un identifiant stable qui garantit la cohérence entre les trois couches. Les fichiers JSONL servent de source de vérité reproductible (générés avec seed fixe), les nœuds Neo4j permettent les requêtes relationnelles, et les endpoints API exposent les données au frontend. Cette architecture assure la traçabilité : toute donnée affichée dans l'interface peut être retracée jusqu'à sa source JSONL.

6.1.8 Pipeline d'analyse ML

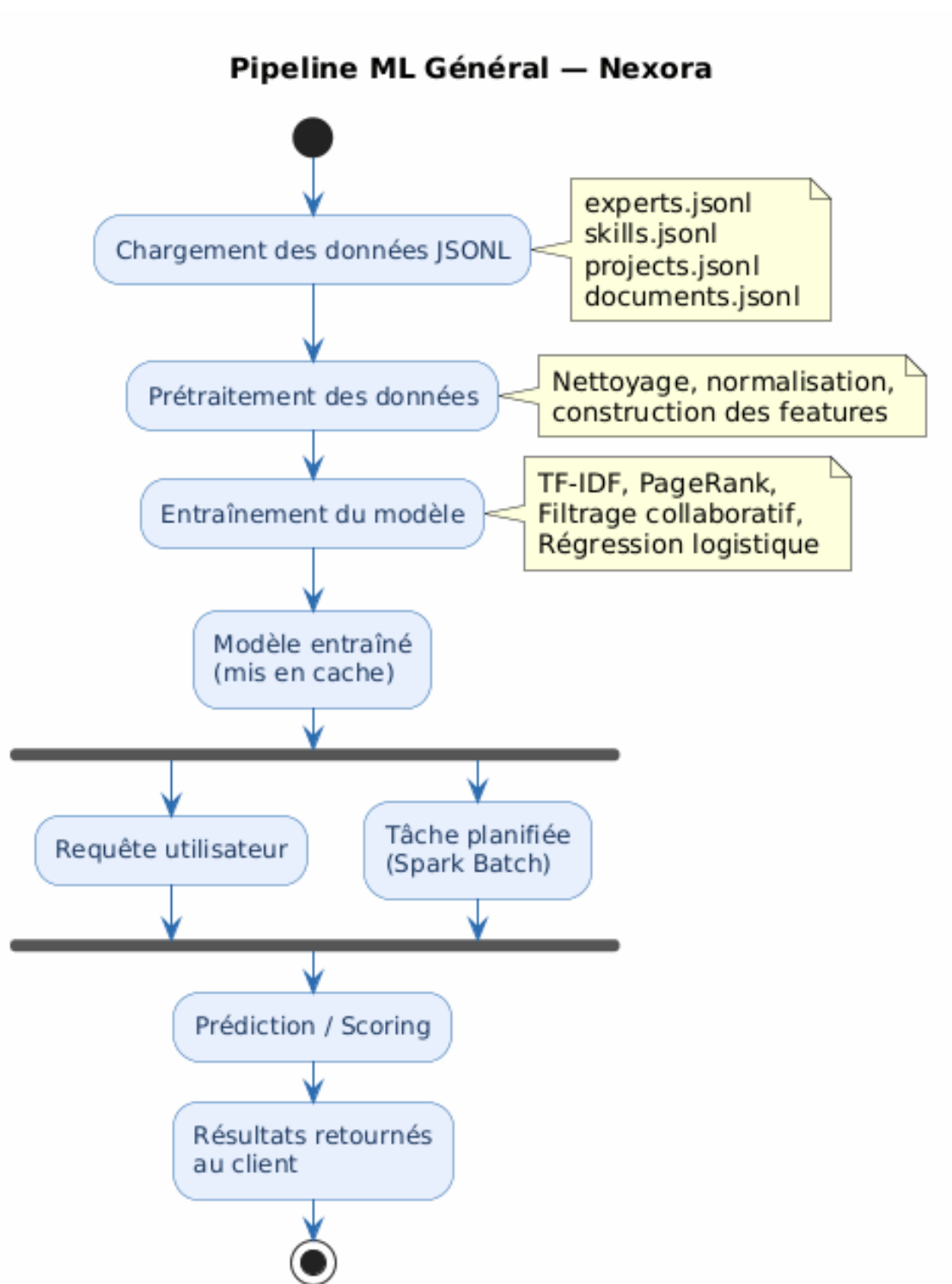


FIGURE 6.8 – Pipeline d'analyse et modules IA (vue d'ensemble)

Le pipeline ML (Figure 6.8) décrit les cinq étapes du traitement IA dans NEXORA.

- (1) **Ingestion** : chargement des fichiers JSONL et création des nœuds/relation Neo4j.
- (2) **Normalisation** : nettoyage des textes, tokenisation et lemmatisation pour les traite-

ments NLP. (3) **Représentation** : calcul des vecteurs TF-IDF à partir des descriptions et titres. (4) **Scoring** : calcul de similarité cosinus pour la recommandation d'experts, PageRank pour la centralité, et classification pour la catégorisation documentaire. (5) **Exposition** : les résultats sont servis via l'API REST. En parallèle, PySpark exécute des agrégations batch dont les sorties complètent les endpoints Big Data. Le fallback rule-based intervient si le LLM (Groq) est indisponible.

6.2 Base de Données Neo4j

6.3 Pourquoi Neo4j ?

Le choix de Neo4j comme base de données principale répond à la nature intrinsèquement relationnelle du problème traité. NEXORA vise à relier des entités hétérogènes (experts, compétences, projets, documents) afin de permettre des recherches relationnelles, des recommandations et des analyses de centralité. Dans ce contexte, la relation constitue un objet de première classe : elle doit être stockée, parcourue et analysée avec un coût maîtrisé.

Neo4j est adapté à ce besoin grâce à l'**index-free adjacency**, qui permet de traverser des relations de manière native. En pratique, les opérations de voisinage (par exemple, experts partageant des compétences), les chemins (shortest path), ou les motifs de graphe (collaborations inter-départements) s'expriment naturellement, sans explosion de jointures comparable à un modèle relationnel.

6.4 Schéma de Données

Le schéma Neo4j de NEXORA a été conçu pour rester lisible, extensible et compatible avec plusieurs usages : recherche d'experts, analyse de compétences, recommandations et visualisation 3D.

TABLE 6.1 – Types de nœuds dans Neo4j

Label	Propriétés (exemples)	Rôle dans le système
Expert	id, name, email, department, location, experience_years	Entité centrale ; support de ranking, recherche
Skill	id, name, category, demand	Normalisation des compétences ; matching, lacunes
Project	id, name, status, date, tech_stack	Contexte d'activité ; demande de compétences
Document	id, title, type, topic, rating, views	Capital de connaissances ; recherche documentaire

6.5 Statistiques de la Base de Données

TABLE 6.2 – Statistiques de la base (ordres de grandeur)

Indicateur	Valeur / observation
Employés (JSONL preset large)	2000+
Compétences	80+ (selon catégories)
Projets (preset medium)	120
Documents (preset medium)	500
Catalogues additionnels	companies.jsonl, technologies.jsonl

6.6 Backend FastAPI

6.7 Présentation de FastAPI

FastAPI est un framework web Python moderne conçu pour construire des API robustes, performantes et typées. Son intérêt majeur réside dans l'association entre le typage Python, la validation via Pydantic et la génération automatique de documentation OpenAPI. Dans NEXORA, cette caractéristique est déterminante.

FastAPI apporte également un modèle d'exécution compatible ASGI, ce qui rend naturel l'usage d'endpoints asynchrones et de WebSocket. La composition via **Depends** facilite l'injection de comportements transverses (authentification, autorisation, accès base, rate

limiting), sans dupliquer du code dans chaque route. Enfin, l'organisation en routeurs correspond bien à la modularité du projet : chaque domaine fonctionnel peut évoluer sans impacter les autres.

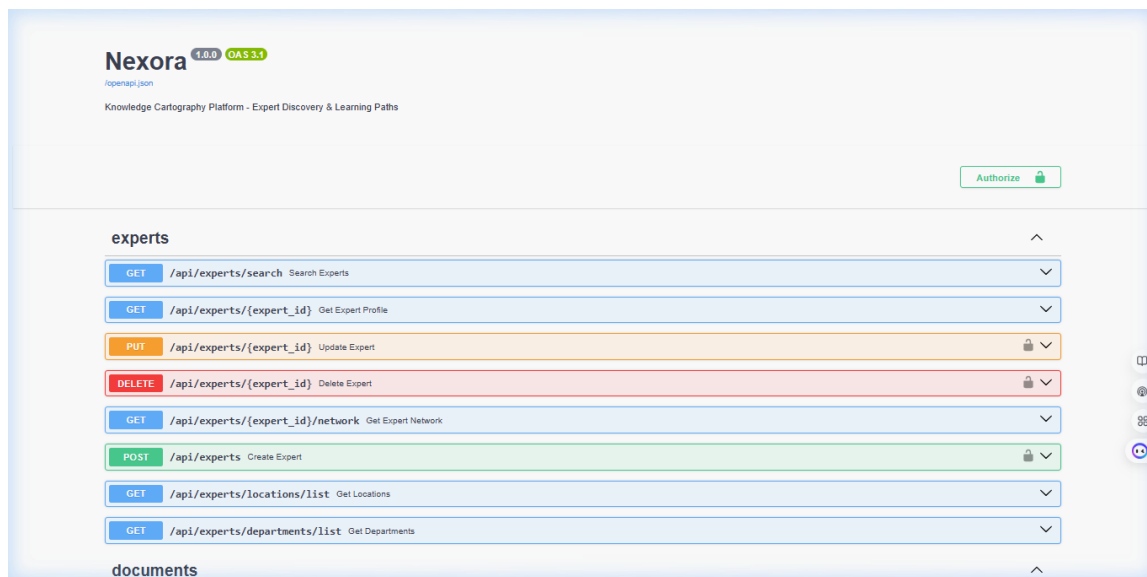


FIGURE 6.9 – Documentation interactive de l'API (Swagger / OpenAPI)

La Figure 6.10 montre la documentation générée automatiquement par OpenAPI. Elle permet de valider rapidement les contrats d'API, de tester les endpoints (auth, IA, Big Data) et de faciliter l'intégration frontend grâce à une référence unique et versionnée.

La documentation Swagger fournit :

- **Liste des endpoints** : tous les routes disponibles organisées par tag
- **Schémas de requête** : structures JSON attendues pour chaque endpoint
- **Schémas de réponse** : structures JSON retournées par chaque endpoint
- **Interface de test** : possibilité d'exécuter des requêtes directement depuis le navigateur
- **Exemples d'utilisation** : exemples concrets pour chaque endpoint

Les étapes du flux d'authentification sont :

- **Soumission** : l'utilisateur envoie email et mot de passe via le formulaire
- **Vérification** : le backend vérifie les identifiants contre la base de données
- **Génération JWT** : si valide, un token JWT est généré avec claims (sub, role, exp)
- **Stockage** : le token est stocké côté client (localStorage)
- **Utilisation** : le token est transmis dans l'en-tête Authorization pour les requêtes API
- **Validation** : chaque endpoint valide le token avant d'exécuter la logique métier

6.8 Endpoints API

TABLE 6.3 – Endpoints d’authentification

Endpoint	Méthode	Description
/api/auth/register	POST	Inscription avec validation Pydantic
/api/auth/login	POST	Login JSON + rate limiting
/api/auth/verify	GET	Vérification token
/api/auth/me	GET	Profil courant à partir du JWT

Ces endpoints constituent la base de la sécurité applicative : `/login` délivre un token signé, `/verify` valide sa conformité, et `/me` permet au frontend de récupérer le profil courant sans stocker d’état côté serveur. La présence d’une documentation interactive (Swagger) accélère le test manuel et réduit les erreurs d’intégration : chaque champ attendu et chaque code de retour sont explicités.

L’endpoint `/api/auth/register` permet la création de nouveaux comptes utilisateur avec une validation stricte des données d’entrée via Pydantic. Les champs obligatoires incluent l’email (vérifié pour l’unicité), le mot de passe (hashé avec bcrypt avant stockage), le nom et le rôle par défaut. Cette validation côté serveur empêche l’injection de données malveillantes et garantit l’intégrité de la base d’utilisateurs dès l’inscription.

L’endpoint `/api/auth/login` implémente un mécanisme de rate limiting pour prévenir les attaques par force brute. Après un nombre prédéfini de tentatives échouées depuis la même adresse IP, l’endpoint retourne une erreur 429 (Too Many Requests) pendant une période de refroidissement. Cette protection est essentielle pour sécuriser le point d’entrée principal du système et protéger les comptes utilisateur contre les tentatives d’intrusion automatisées.

L’endpoint `/api/auth/verify` est utilisé par le frontend pour vérifier périodiquement la validité du JWT stocké côté client. Cette vérification permet de détecter les tokens expirés et de rediriger l’utilisateur vers la page de connexion avant que les requêtes API ne commencent à échouer. Cette approche proactive améliore l’expérience utilisateur en évitant les erreurs inattendues lors de l’utilisation de l’application.

L’endpoint `/api/auth/me` retourne le profil complet de l’utilisateur authentifié (nom, email, rôle, département, compétences) en extrayant ces informations directement du JWT sans requêter la base de données. Cette optimisation réduit la charge sur la base de données et améliore les performances pour les endpoints qui ont besoin d’afficher les informations utilisateur dans l’interface (par exemple, dans la barre de navigation ou les pages de profil).

6.9 Authentification JWT

L'authentification repose sur JWT afin d'assurer un backend stateless. Le token est généré à la connexion après vérification des identifiants et est signé côté serveur afin d'empêcher toute falsification. Il contient les claims essentiels : **sub** (identité), **role** (rôle RBAC) et **exp** (expiration), ce qui permet au backend de décider rapidement des droits d'accès sans maintenir de session côté serveur.

Le processus de génération du JWT utilise une clé secrète (configurable via variable d'environnement **SECRET_KEY**) et l'algorithme de signature HS256. Cette clé doit être gardée confidentielle et ne doit jamais être exposée dans le code source ou les logs. En production, il est recommandé d'utiliser une clé longue et générée aléatoirement, et de la faire circuler via des secrets management (Vault, AWS Secrets Manager, etc.) plutôt que via des variables d'environnement simples.

Le claim **sub** (subject) contient l'identifiant unique de l'utilisateur (typiquement son email ou son ID dans la base de données). Ce claim est utilisé par les gardes d'autorisation pour identifier l'utilisateur courant et récupérer ses informations supplémentaires si nécessaire. Le claim **role** contient le rôle RBAC de l'utilisateur (user, manager, ou admin) et est utilisé directement par les dépendances d'autorisation pour déterminer si l'utilisateur a le droit d'accéder à un endpoint spécifique sans requêter la base de données.

Le claim **exp** (expiration) contient le timestamp d'expiration du token, calculé en ajoutant la durée configurée (30 minutes par défaut) au timestamp actuel. Ce claim est vérifié automatiquement par la bibliothèque de décodage JWT, qui retourne une erreur si le token est expiré. Cette expiration automatique limite la fenêtre d'opportunité pour un attaquant qui aurait intercepté un token valide, et force les utilisateurs à se réauthentifier périodiquement, ce qui améliore la sécurité globale du système.

Dans NEXORA, le frontend transmet ce JWT dans l'en-tête **Authorization: Bearer <token>** lors des appels API. Les gardes d'autorisation appliquent ensuite le contrôle d'accès (user/manager/admin) en fonction du claim **role**. En cas de token manquant, expiré ou invalide, l'API renvoie des erreurs adaptées (401/403) afin de conserver un comportement explicite et sécurisé.

Le stockage du JWT côté client s'effectue généralement dans le `localStorage` du navigateur pour une persistance entre les sessions. Cependant, cette approche présente des risques de sécurité si l'application est vulnérable aux attaques XSS (Cross-Site Scripting). Pour atténuer ce risque, le frontend implémente des mesures de protection : validation stricte des entrées utilisateur, sanitization des données affichées, et utilisation de Content Security Policy (CSP) pour limiter l'exécution de scripts non autorisés. Dans une version de

production, il serait préférable d'utiliser des cookies `HttpOnly` avec l'attribut `SameSite` pour stocker le token.

Les gardes d'autorisation sont implémentés sous forme de dépendances FastAPI qui sont appliquées aux endpoints sensibles. Le garde `require_auth` vérifie simplement que le token est présent et valide, tandis que `require_manager` vérifie en plus que le rôle est manager ou admin, et `require_admin` vérifie que le rôle est admin. Ces gardes peuvent être composés : par exemple, un endpoint peut nécessiter `require_auth` pour vérifier l'authentification, puis effectuer des vérifications supplémentaires basées sur l'identité de l'utilisateur (par exemple, vérifier qu'un utilisateur ne peut modifier que ses propres données).

La durée d'expiration est configurable et vaut 30 minutes par défaut, ce qui limite la fenêtre d'usage d'un token compromis. Cette approche est complétée par des pratiques de durcissement (hash des mots de passe, limitation de tentatives de login et séparation claire des endpoints publics/privés). Les mots de passe sont hashés avec `bcrypt`, un algorithme de hachage lent et adapté spécifiquement aux mots de passe, ce qui rend les attaques par force brute ou par dictionnaire extrêmement coûteuses en temps de calcul.

6.10 Frontend React

6.11 Architecture du Frontend

Le frontend de NEXORA est une **SPA** construite avec React 18 et TypeScript. Cette architecture permet une navigation fluide, des composants réutilisables et une modularité adaptée aux nombreux modules fonctionnels.

Le frontend contient **17 pages principales** (dont Login/Register et une page Forbidden). Certaines pages sont réservées aux rôles élevés via `ProtectedRoute`.

La structure s'appuie sur un routage centralisé et des composants transverses (navigation, layout, gestion du thème). Le typage TypeScript contribue à la fiabilité des échanges avec l'API (contrats, DTO) et limite les erreurs d'exécution. Sur le plan UX, la cohérence est renforcée par l'utilisation de MUI et par la séparation claire entre pages (fonctionnalités) et composants (réutilisables).

L'utilisation de React 18 apporte des améliorations significatives en termes de performance grâce au mode concurrent et au `automatic batching`, qui regroupe les mises à jour d'état pour réduire les re-rendus inutiles. Les hooks personnalisés permettent d'encapsuler la logique réutilisable (par exemple, `useAuth` pour la gestion de l'authentification, `useApi` pour les appels API, `useWebSocket` pour les connexions temps réel), ce qui améliore la maintenabilité du code et facilite les tests unitaires.

TypeScript apporte une sécurité typage statique qui détecte les erreurs potentielles dès la phase de développement. Les interfaces TypeScript définissent les contrats avec l'API backend, garantissant que les structures de données sont cohérentes entre le frontend et le backend. Par exemple, l'interface `User` définit les propriétés attendues pour un utilisateur (id, name, email, role, etc.), et TypeScript vérifie que toutes les utilisations de cette interface respectent ces propriétés, réduisant ainsi les erreurs d'exécution liées à des propriétés manquantes ou mal typées.

Le routing est géré par React Router, qui permet une navigation déclarative et des transitions fluides entre les pages sans rechargement complet du navigateur. Les routes sont protégées par le composant `ProtectedRoute`, qui vérifie la présence d'un JWT valide et redirige vers la page de connexion si nécessaire. Les routes sensibles (administration, gestion des utilisateurs) utilisent des gardes supplémentaires pour vérifier que l'utilisateur dispose des rôles appropriés (manager ou admin).

L'écran de connexion comprend :

- **Formulaire d'authentification** : champs email et mot de passe sécurisés
- **Validation** : vérification du format email et mot de passe minimum
- **Messages d'erreur** : affichage clair en cas d'échec d'authentification
- **Bouton de connexion** : déclenche la requête d'authentification
- **Liens utiles** : option d'inscription ou récupération de mot de passe

6.12 Visualisation 3D

La visualisation 3D, basée sur `react-force-graph-3d`, constitue un élément différenciant de NEXORA. Elle représente les nœuds et liens du graphe dans un espace interactif, avec des interactions (zoom, rotation, survol) et des filtres.

Cette visualisation vise à rendre tangible la structure relationnelle : l'utilisateur peut naviguer entre experts, compétences, projets et documents, puis comprendre les proximités (voisinages) et les ponts entre communautés. Pour préserver la performance, l'UI privilégie des sous-graphes et des filtres, plutôt qu'un affichage exhaustif. L'objectif n'est pas de visualiser **tout**, mais de fournir un outil d'exploration guidée et exploitable dans un contexte décisionnel.

La bibliothèque `react-force-graph-3d` utilise WebGL pour le rendu 3D, ce qui permet d'afficher des milliers de nœuds et liens avec des performances satisfaisantes sur les navigateurs modernes. Les interactions utilisateur (zoom, rotation, pan) sont gérées par la bibliothèque et permettent une navigation intuitive dans l'espace 3D. Les nœuds sont colorés selon leur type (experts en bleu, compétences en vert, projets en orange, documents

en violet) et leur taille peut varier en fonction de métriques comme le degré de centralité ou le nombre de connexions.

Les filtres permettent à l'utilisateur de restreindre l'affichage à un sous-graphe pertinent : par exemple, filtrer par département pour visualiser uniquement les experts d'un service, ou filtrer par compétence pour identifier les experts possédant une compétence spécifique. Ces filtres sont appliqués côté backend via des requêtes Cypher optimisées, ce qui réduit la quantité de données transférées et améliore les performances. L'utilisateur peut également sélectionner un nœud spécifique pour afficher ses détails (nom, compétences, projets associés) dans un panneau latéral, ce qui facilite l'exploration contextuelle.

La visualisation 3D est particulièrement utile pour identifier des patterns organisationnels invisibles dans les vues tabulaires traditionnelles : par exemple, on peut repérer des experts qui font le pont entre différents départements (connecteurs), des compétences émergentes qui sont demandées dans plusieurs projets, ou des communautés de pratique regroupées autour de thématiques spécifiques. Ces insights peuvent guider les décisions RH en matière de formation, de mobilité interne ou de constitution d'équipes pluridisciplinaires.

Les fonctionnalités de la visualisation 3D incluent :

- **Navigation 3D** : zoom, rotation, pan pour explorer le graphe
- **Sélection de nœuds** : clic pour afficher les détails d'un expert/compétence/projet
- **Filtrage** : filtres par département, compétence, type de nœud
- **Coloration** : codes couleur par type d'entité pour une lecture rapide
- **Taille dynamique** : taille des nœuds basée sur la centralité ou le degré
- **Légende interactive** : affichage des types et légende du graphe

6.13 Présentation de la Plateforme

6.13.1 Page de Connexion

La page de connexion constitue la porte d'entrée du système et matérialise l'application du contrôle d'accès. L'utilisateur saisit un identifiant et un mot de passe, puis l'API valide les informations, applique le rate limiting si nécessaire et renvoie un JWT.

La Figure ?? présente des indicateurs synthétiques (activité, tendances, répartitions) destinés à donner une lecture rapide de l'état du système. Ces visualisations exploitent les endpoints backend et, lorsque nécessaire, les sorties Big Data.

Le dashboard principal affiche :

- **Statistiques générales** : nombre total d'experts, compétences, projets, documents
- **Graphiques d'activité** : activité récente, tendances d'utilisation
- **Répartitions** : distribution par département, par compétence, par rôle
- **Alertes** : notifications d'événements importants ou anomalies
- **Accès rapides** : raccourcis vers les fonctionnalités principales

6.13.2 Recherche d'Experts

La recherche d'experts s'appuie sur un moteur sémantique (TF-IDF + similarité cosinus) exposé par `POST /api/ai/recommend-experts`. L'approche consiste à représenter le contenu textuel sous forme de vecteurs, puis à classer les experts par score de similarité.

L'interface de recommandation d'experts comprend :

- **Barre de recherche** : champ de saisie pour la requête en langage naturel
- **Filtres** : filtres par département, compétence, disponibilité
- **Liste de résultats** : experts classés par score de pertinence
- **Détails de chaque expert** : nom, compétences, projets, score de similarité
- **Explication du score** : correspondances avec la requête, facteurs de ranking
- **Actions rapides** : contacter, voir le profil, ajouter aux favoris

6.13.3 Chatbot Veda

Veda constitue un assistant conversationnel intégré, exposé via `POST /api/ai/chatbot`. L'architecture est **LLM-first** : si la clé Groq est présente, le backend interroge le modèle ; en l'absence de clé ou en cas d'erreur, le système bascule sur un moteur rule-based.

La Figure ?? illustre l'expérience conversationnelle. Le chatbot fournit un accès naturel à la connaissance interne ; en cas d'indisponibilité du LLM, le backend applique un repli afin de maintenir une réponse utilisable.

L'interface du chatbot Veda inclut :

- **Zone de conversation** : historique des échanges utilisateur/chatbot
- **Champ de saisie** : zone de texte pour poser des questions
- **Indicateur de statut** : affichage du mode (LLM ou rule-based)
- **Suggestions** : questions suggérées pour guider l'utilisateur
- **Formatage des réponses** : réponses structurées avec listes, tableaux si nécessaire
- **Historique** : conservation de l'historique de conversation

Les fonctionnalités de messagerie incluent :

- **Communication temps réel** : échanges instantanés via WebSocket
- **Indicateur de saisie** : affichage quand l'autre utilisateur est en train d'écrire
- **Historique** : conservation de l'historique des conversations
- **Notifications** : alertes pour nouveaux messages
- **Groupes** : conversations de groupe pour les équipes ou projets
- **Fichiers** : possibilité d'envoyer des fichiers et documents

Le Team Builder permet de :

- **Définir les besoins** : spécifier les compétences requises pour un projet
- **Rechercher des candidats** : trouver des experts correspondant aux besoins
- **Évaluer la pertinence** : scores de correspondance basés sur compétences et disponibilité
- **Constituer l'équipe** : sélectionner les membres proposés
- **Vérifier l'équilibre** : analyser la composition de l'équipe (compétences, rôles)
- **Exporter** : générer une proposition d'équipe pour validation

Les fonctionnalités du workspace d'équipe incluent :

- **Vue de projet** : informations sur le projet, objectifs et livrables
- **Liste des membres** : équipe assignée avec rôles et responsabilités
- **Compétences requises** : compétences nécessaires pour le projet
- **Messagerie de groupe** : canal de communication dédié à l'équipe
- **Suivi de tâches** : gestion des tâches et leur statut
- **Documents partagés** : accès aux documents liés au projet

6.13.4 Documentation API (Swagger)

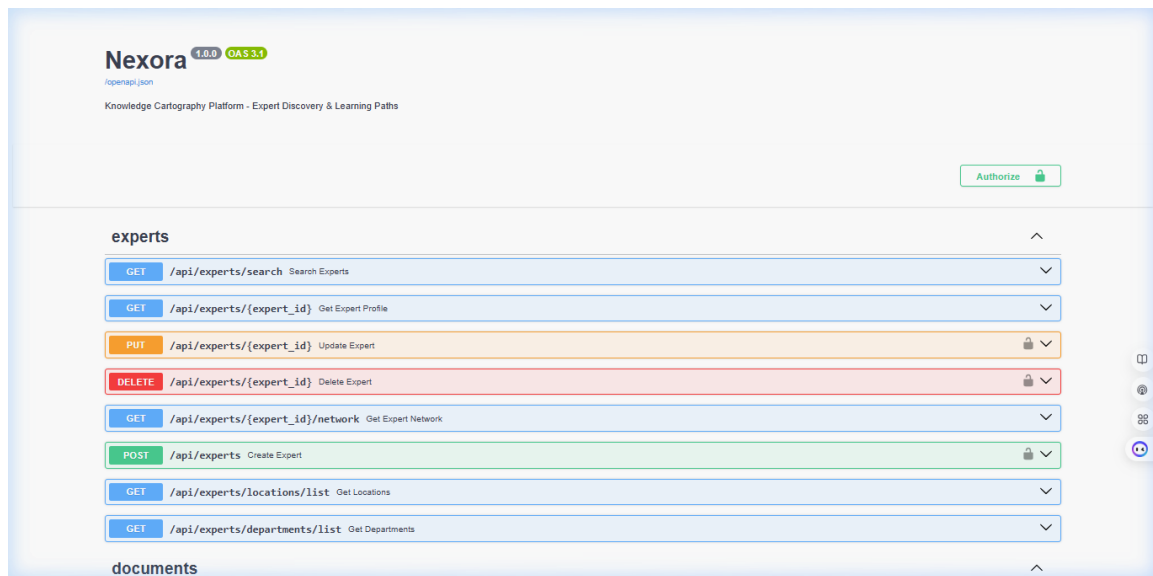


FIGURE 6.10 – Documentation interactive de l'API (Swagger / OpenAPI)

La Figure 6.10 montre la documentation générée automatiquement par OpenAPI, permettant de valider les contrats d'API et de tester les endpoints.

6.13.5 Autres Interfaces

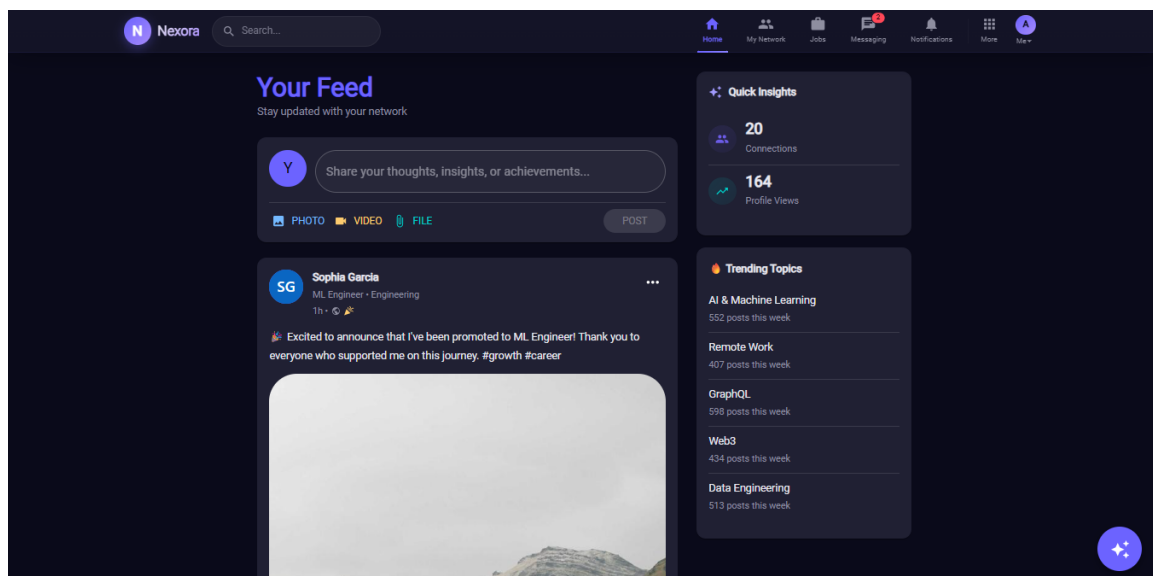


FIGURE 6.11 – Fil d'actualité (Feed)

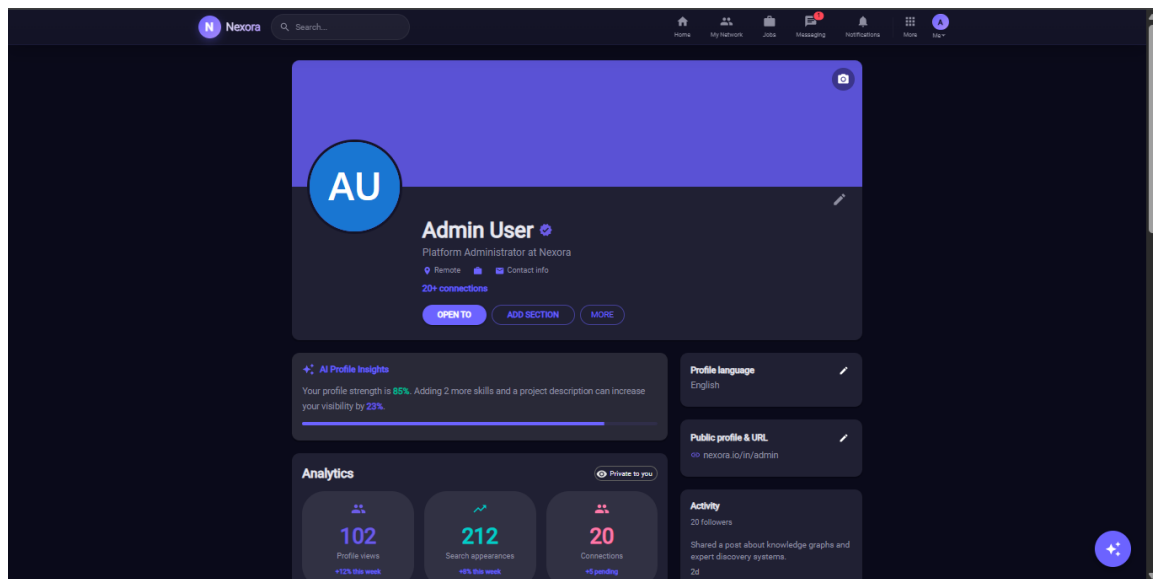


FIGURE 6.12 – Page de profil utilisateur

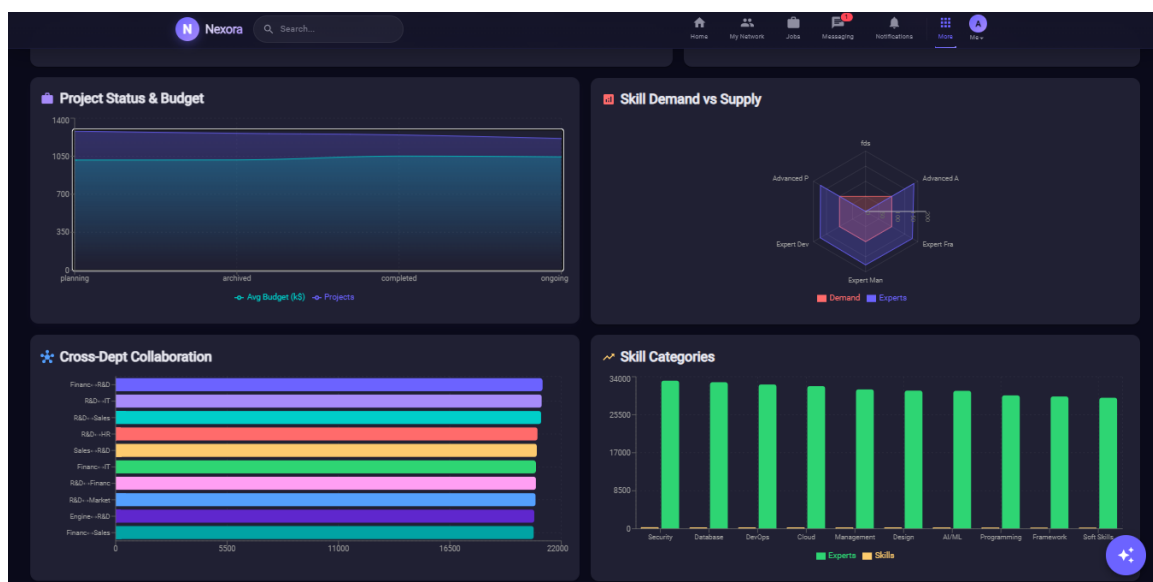


FIGURE 6.13 – Tableau de bord — vue complémentaire

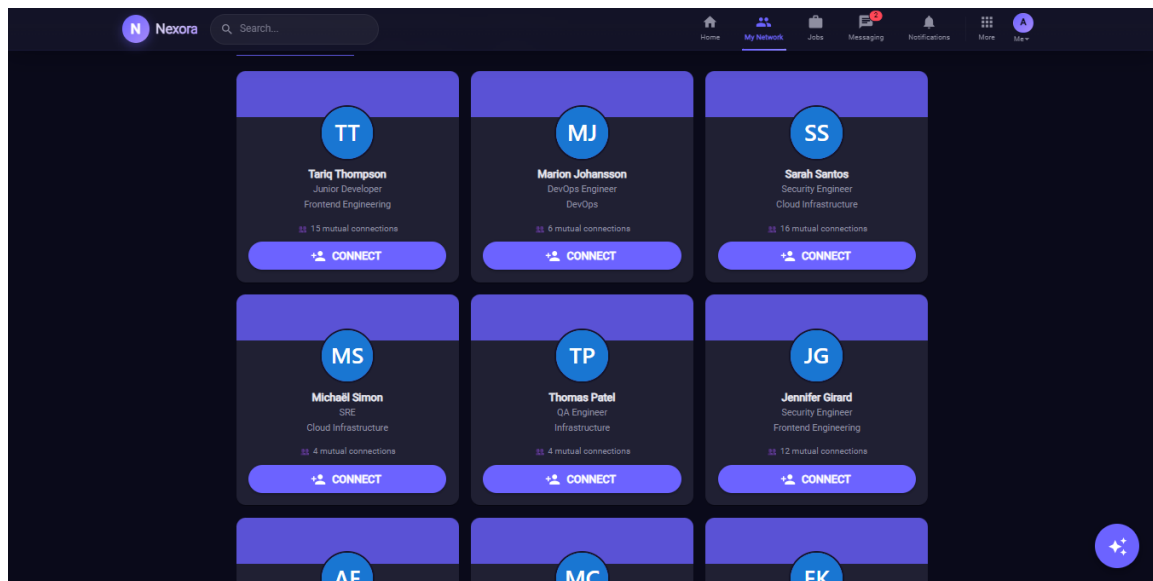


FIGURE 6.14 – Réseau de connexions

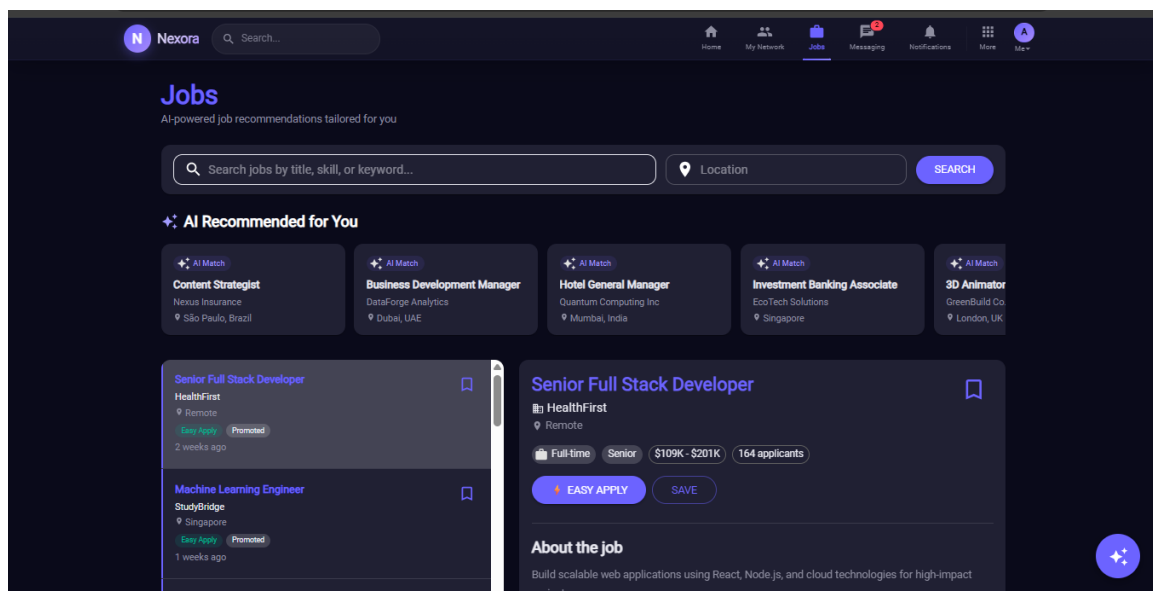


FIGURE 6.15 – Offres et opportunités

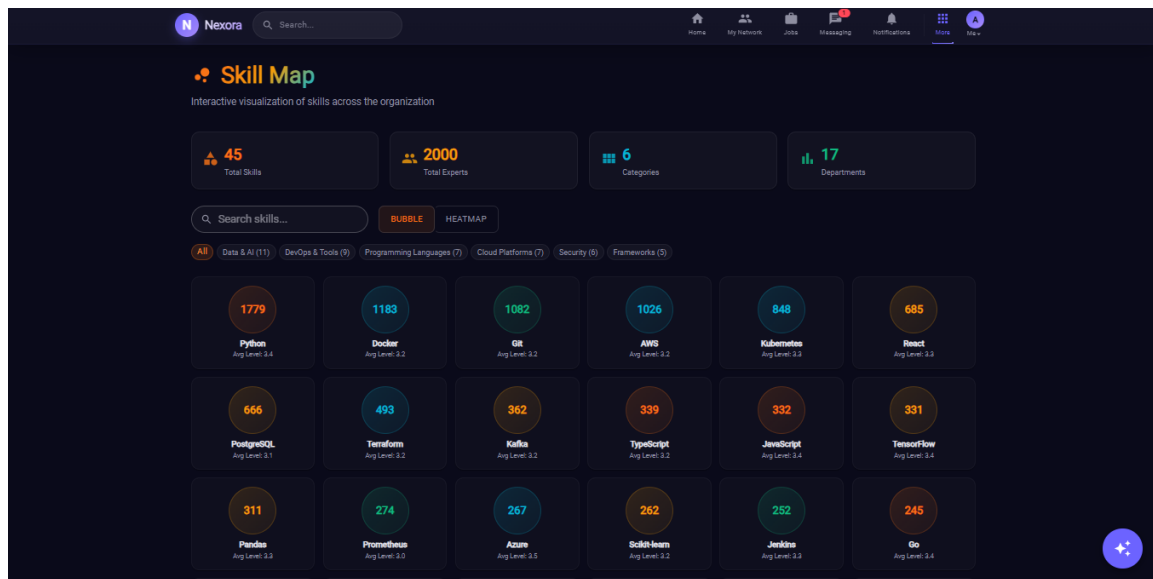


FIGURE 6.16 – Cartographie des compétences (Skill Map)

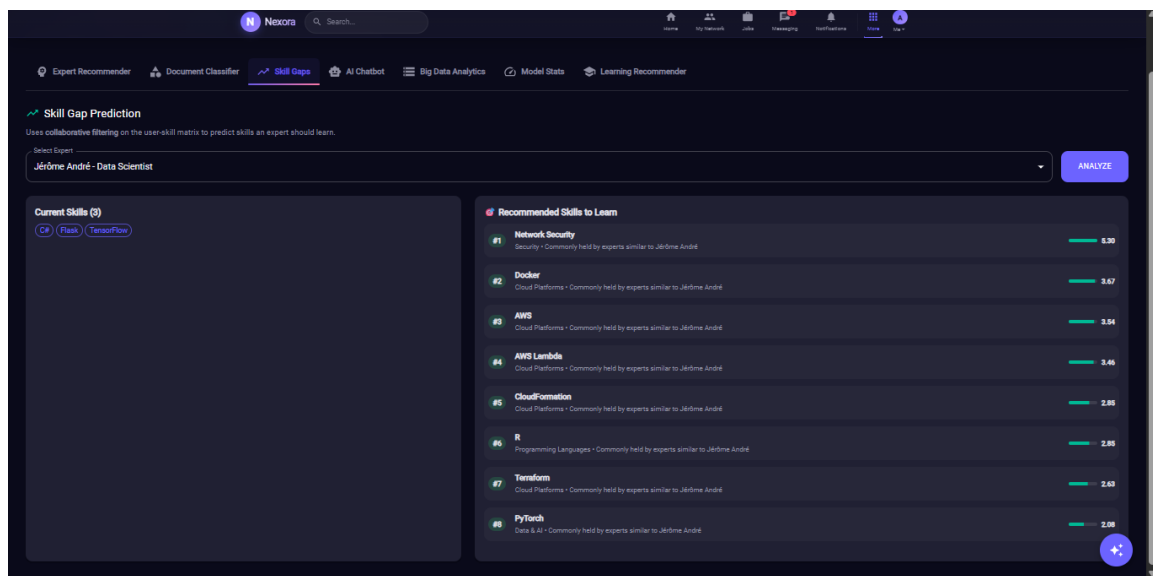


FIGURE 6.17 – Analyse des lacunes de compétences (Skill Gap)

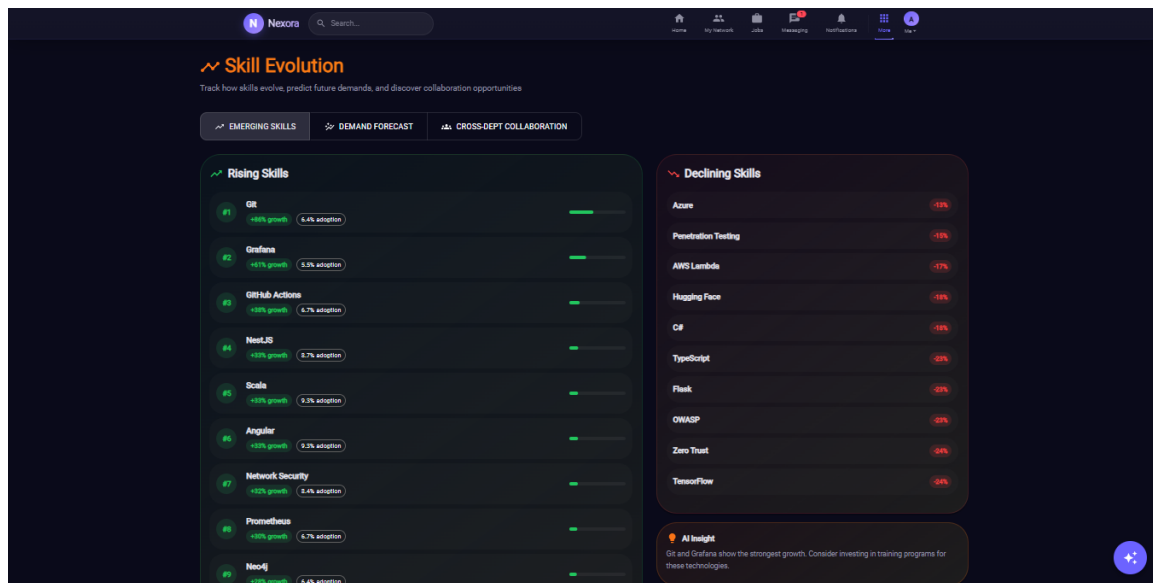


FIGURE 6.18 – Évolution des compétences

6.14 Conclusion

Les captures d'écran présentées couvrent l'ensemble du périmètre fonctionnel de NEXORA : de l'authentification à la collaboration en temps réel, en passant par la recherche sémantique, l'assistant conversationnel et l'administration.

Chapitre 7

Données et Déploiement

7.1 Guide d’Installation

7.2 Prérequis

L’installation de NEXORA repose prioritairement sur Docker. Configuration minimale : 4 Go RAM (recommandée : 8 Go). Ports requis : 3000 (frontend), 8000 (API), 7474 et 7687 (Neo4j).

Cette approche réduit fortement la complexité d’installation : l’utilisateur n’a pas besoin d’installer manuellement Neo4j, Node.js ou un environnement Python complet. Les services sont isolés et démarrent dans un ordre contrôlé grâce aux healthchecks, ce qui limite les erreurs de dépendances. En complément, un fichier `.env` centralise les paramètres (URI Neo4j, clé JWT, clé Groq) et permet d’adapter rapidement l’environnement.

Le fichier `docker-compose.yml` définit l’orchestration complète des services. Chaque service est configuré avec son propre réseau Docker pour assurer l’isolation et la sécurité des communications internes. Les volumes partagés permettent de persister les données Neo4j et de faciliter le développement en montant les fichiers source locaux dans les conteneurs. Les healthchecks sont configurés pour vérifier que chaque service est opérationnel avant de démarrer les services dépendants : par exemple, le backend attend que Neo4j réponde correctement aux requêtes de santé avant de s’initialiser, et le frontend attend que le backend soit accessible avant de servir l’application.

L’installation nécessite simplement Docker et Docker Compose installés sur la machine hôte. Une fois ces prérequis satisfaits, l’utilisateur clone le dépôt, configure le fichier `.env` avec ses propres clés et paramètres, puis exécute `docker-compose up -build`. Cette commande construit les images Docker (si nécessaire), démarre tous les services dans l’ordre approprié et expose les ports configurés. L’ensemble du processus prend généralement

moins de 10 minutes sur une machine moderne disposant d’une connexion internet pour le téléchargement des images de base.

En cas de problème lors du démarrage, les logs de chaque service sont accessibles via `docker-compose logs <service>`, ce qui facilite le diagnostic. Les erreurs les plus courantes incluent des ports déjà occupés (conflit avec d’autres applications), des permissions insuffisantes sur les volumes partagés, ou des configurations incorrectes dans le fichier `.env`. Le guide d’installation fournit des solutions pour ces scénarios, ainsi que des commandes de nettoyage (`docker-compose down -v`) pour réinitialiser l’environnement si nécessaire.

L’architecture de déploiement comprend :

- **Service Neo4j** : base de données graphe avec healthcheck
- **Service Backend** : API FastAPI avec dépendances vers Neo4j
- **Service Frontend** : application React servie via Nginx
- **Réseaux Docker** : isolation des communications entre services
- **Volumes** : persistance des données Neo4j et montage des sources
- **Healthchecks** : vérification de santé avant démarrage des dépendants
- **Ports exposés** : 3000 (frontend), 8000 (API), 7474/7687 (Neo4j)

7.3 Variables d’Environnement

TABLE 7.1 – Variables d’environnement principales

Variable	Description
NEO4J_URI	URI de connexion Neo4j
SECRET_KEY	Clé secrète de signature JWT
ACCESS_TOKEN_EXPIREDMINUTES	Validité du token (30 min par défaut)
GROQ_API_KEY	Clé API Groq (Veda LLM)

7.4 Données du Système

7.5 Format des Données

Les données sont fournies au format **JSONL** (JSON Lines), adapté aux traitements batch PySpark et aux scénarios de streaming.

Le format JSONL offre un compromis efficace entre simplicité et scalabilité : chaque ligne est un objet autonome, ce qui facilite l’ingestion incrémentale, le traitement distribué et le

débogage. Dans NEXORA, ce choix permet de partager un jeu de données reproductible (généré avec seed) et de l'utiliser à la fois pour l'alimentation du graphe Neo4j, pour les traitements ML (features textuelles) et pour les agrégations Big Data.

Contrairement au JSON traditionnel qui nécessite de charger l'intégralité du fichier en mémoire pour le parser, le format JSONL permet de traiter les données ligne par ligne (streaming), ce qui est particulièrement adapté pour les jeux de données volumineux. Chaque ligne peut être lue, validée et traitée indépendamment, ce qui facilite le parallélisme et la résilience aux erreurs : si une ligne est corrompue, elle peut être ignorée sans compromettre le traitement du reste du fichier. Cette propriété est essentielle pour les traitements PySpark, qui distribuent le traitement sur plusieurs nœuds et doivent gérer les partitions de manière autonome.

Les fichiers JSONL de NEXORA sont générés par un script Python paramétrable qui accepte différents presets de taille (small, medium, large). Ce script utilise une seed fixe pour garantir la reproductibilité : deux exécutions avec les mêmes paramètres produiront exactement le même jeu de données. Les données sont générées avec des contraintes de cohérence sémantique : par exemple, les compétences assignées aux employés sont choisies en fonction de leur département (un développeur aura plus de chances d'avoir des compétences techniques qu'un commercial), et les projets incluent des employés dont les compétences correspondent aux besoins du projet. Cette cohérence sémantique est cruciale pour que les algorithmes de recommandation produisent des résultats pertinents et que la visualisation du graphe soit interprétable.

Le script de génération de données produit également des fichiers de métadonnées qui documentent les statistiques du jeu de données (nombre d'employés, distribution des compétences, nombre de projets, etc.). Ces métadonnées sont utilisées pour valider que l'ingestion dans Neo4j s'est déroulée correctement et pour fournir des informations contextuelles dans l'interface (par exemple, afficher le nombre total d'experts dans le système). Les fichiers JSONL peuvent également être utilisés pour créer des jeux de données de test spécifiques (par exemple, un jeu de données contenant uniquement des employés du département "Engineering" pour tester des fonctionnalités spécifiques).

TABLE 7.2 – Fichiers de données JSONL

Fichier	Type	Description
employees.jsonl	RH	Employés, attributs, compétences imbriquées
skills.jsonl	Référentiel	Catalogue de compétences
projects.jsonl	Projets	Projets, statuts, compétences requises
documents.jsonl	Connaissance	Documents, topics, auteurs

Le modèle de données de NEXORA repose sur :

- **Entités principales** : Employee, Skill, Project, Document, Company, Technology
- **Relations sémantiques** : HAS_SKILL, WORKS_ON, CONTRIBUTED_TO, REQUIRES_SKILL
- **Attributs riches** : propriétés détaillées pour chaque entité (nom, email, département, etc.)
- **Identifiants stables** : clés uniques garantissant la cohérence entre sources
- **Extensibilité** : possibilité d'ajouter de nouveaux types sans casser l'existant

Sur le plan sémantique, le modèle vise à représenter des liens utiles à l'analyse : un expert peut *posséder* ou *maîtriser* des compétences, *participer* à des projets, et *contribuer* à des documents. Ces liens permettent d'exprimer des requêtes Cypher de type « trouver des experts proches d'un besoin », « expliquer une recommandation par un chemin » ou « détecter des ponts inter-départements ».

Les requêtes types supportées par le modèle incluent :

- **Recherche d'experts** : trouver des experts par compétences, projets, département
- **Analyse de centralité** : identifier les experts les plus connectés (PageRank)
- **Détection de communautés** : regrouper les experts par affinités de compétences
- **Ponts inter-départements** : identifier les experts qui connectent différents services
- **Parcours de recommandation** : expliquer une recommandation par un chemin dans le graphe

Sur le plan technique, les identifiants et attributs portés par les objets (ex. `id`, `department`, `category`, `topic`) garantissent la cohérence entre les sources. Les mêmes clés servent :

- à charger les données depuis JSONL et à vérifier les correspondances lors de l'ingestion
- à indexer les nœuds (pour accélérer la recherche et limiter le coût des traversées)
- à alimenter les traitements ML (TF-IDF sur les descriptions/titres, classification par features textuelles, scoring de centralité via PageRank)

- à construire des sous-graphes affichables dans la visualisation 3D

Cette cohérence technique permet de :

- Garantir l'intégrité des données entre JSONL et Neo4j
- Optimiser les performances des requêtes Cypher via les index
- Faciliter les calculs ML avec des attributs structurés
- Assurer une visualisation cohérente et interprétable

Enfin, le modèle est conçu pour rester extensible : de nouveaux types (formations, expériences, technologies, certifications) peuvent être ajoutés sans casser les usages existants, à condition de conserver des identifiants stables et des relations explicites. C'est ce choix qui rend possible l'explicabilité : une recommandation n'est pas seulement un score, elle peut être justifiée par des voisinages (compétences partagées), des contextes (projets) et des traces (documents).

Les principes d'extensibilité du modèle sont :

- **Identifiants stables** : les clés ne changent pas pour assurer la continuité
- **Relations explicites** : chaque lien est clairement défini et typé
- **Schéma évolutif** : ajout de nouveaux types sans migration lourde
- **Explicabilité** : chaque recommandation peut être justifiée par un chemin
- **Rétrocompatibilité** : les anciennes requêtes restent valides après extension

L'organisation des classes suit les principes suivants :

- **Modèles Pydantic** : schémas de validation pour entrées/sorties
- **Routeurs FastAPI** : endpoints API organisés par domaine fonctionnel
- **Services métier** : logique métier isolée et réutilisable
- **Accès données** : abstraction pour Neo4j et JSONL
- **Utilitaires** : fonctions partagées (auth, cache, logging)

Dans NEXORA, cette séparation est importante pour maintenir l'évolutivité : les modèles Pydantic encadrent les entrées/sorties (validation, sérialisation), les routeurs FastAPI portent les contrats d'API, tandis que les services et utilitaires encapsulent les traitements (scoring, fallback, intégration Groq, calculs Big Data). Concrètement, cette structuration réduit les effets de bord : modifier une règle de sécurité ou une stratégie de recommandation ne nécessite pas de réécrire l'interface utilisateur, et inversement. Le diagramme sert donc de support de compréhension et de maintenance, en particulier lorsqu'on ajoute de nouveaux endpoints ou de nouvelles entités.

Les avantages de cette séparation incluent :

- **Maintenabilité** : chaque couche peut être modifiée indépendamment
- **Testabilité** : tests unitaires isolés par couche

- **Réutilisabilité** : services peuvent être réutilisés dans différents contextes
- **Clarté** : responsabilités claires pour chaque composant
- **Évolutivité** : ajout de nouvelles fonctionnalités sans impact sur l'existant

7.6 Sécurité et Performance

7.6.1 Authentification et Autorisation

La sécurité repose sur JWT (stateless) et RBAC. Des gardes centralisées (`require_auth`, `require_manager`, `require_admin`) appliquent les droits. Cette combinaison permet de séparer clairement l'identification (preuve d'identité via le token) et l'autorisation (décision d'accès via le rôle).

Concrètement, les endpoints sensibles imposent une authentification valide, puis vérifient le niveau de privilège requis. Cela se traduit par des réponses explicites (401 si non authentifié, 403 si authentifié mais non autorisé). Côté interface, les pages et actions sont également filtrées pour réduire les erreurs d'usage, mais l'autorité reste le backend : même si un utilisateur manipule l'UI, les gardes empêchent l'accès aux opérations réservées.

La séparation entre identification et autorisation est un principe fondamental de sécurité. L'identification consiste à vérifier que l'utilisateur est bien celui qu'il prétend être (via le JWT), tandis que l'autorisation consiste à vérifier que cet utilisateur a le droit d'effectuer l'action demandée (via le rôle RBAC). Cette séparation permet une gestion fine des permissions : par exemple, un manager peut accéder aux fonctionnalités de staffing (TeamBuilder) mais ne peut pas administrer les utilisateurs, tandis qu'un admin dispose de tous les droits. Les gardes d'autorisation sont appliqués au niveau des endpoints FastAPI, ce qui garantit que toute tentative d'accès non autorisé est bloquée avant même que la logique métier ne soit exécutée.

Le JWT stateless présente l'avantage de ne pas nécessiter de stockage de session côté serveur, ce qui facilite la scalabilité horizontale : plusieurs instances du backend peuvent être déployées derrière un load balancer sans avoir à synchroniser les sessions. Cependant, cela implique que les tokens ne peuvent pas être révoqués facilement avant leur expiration. Pour atténuer ce risque, la durée de vie des tokens est limitée (30 minutes par défaut) et les actions sensibles (suppression de données, modification de rôles) peuvent nécessiter une ré-authentification ou une confirmation supplémentaire. En production, il serait recommandé d'implémenter une liste de révocation (token blacklist) ou d'utiliser des refresh tokens pour améliorer la gestion des sessions.

Les mots de passe sont hashés avec bcrypt avant d'être stockés dans Neo4j. Bcrypt est un algorithme de hachage lent et adapté spécifiquement aux mots de passe : il incorpore un sel (salt) unique pour chaque mot de passe et un facteur de coût (cost factor) qui augmente le temps de calcul nécessaire pour le hachage. Cette caractéristique rend les attaques par force brute ou par dictionnaire extrêmement coûteuses en temps de calcul, ce qui protège les mots de passe même si la base de données est compromise. Le hash

bcrypt est vérifié lors de la connexion en comparant le hash stocké avec le hash calculé à partir du mot de passe fourni, sans jamais stocker ni transmettre le mot de passe en clair.

TABLE 7.3 – Matrice RBAC (synthèse)

Fonctionnalité	User	Manager	Admin
Accès Feed / Profil / Graph / Learning	✓	✓	✓
IA (reco, classification, Veda)	✓	✓	✓
TeamBuilder (staffing)		✓	✓
Administration utilisateurs			✓

Le panneau d'administration permet de :

- **Gérer les utilisateurs** : lister, créer, modifier, supprimer des comptes
- **Gérer les rôles** : assigner ou modifier les rôles RBAC (user/manager/admin)
- **Voir les statistiques** : statistiques d'utilisation et d'activité
- **Auditer les actions** : journal des actions sensibles effectuées
- **Configurer le système** : paramètres globaux et configurations

Le système de notifications inclut :

- **Notifications temps réel** : alertes instantanées via WebSocket
- **Centre de notifications** : historique des notifications consultable
- **Types de notifications** : messages, mentions, mises à jour, alertes système
- **Préférences** : configuration des types de notifications reçues
- **Indicateurs de lecture** : marquage des notifications comme lues/non lues
- **Actions rapides** : possibilité d'agir directement depuis la notification

Chapitre 8

Conclusion et Perspectives

8.1 Bilan du Projet

NEXORA a permis de réaliser une plateforme complète combinant backend FastAPI, graphe Neo4j, frontend React moderne et couche IA/Big Data. Le système intègre la recherche sémantique, la classification documentaire, des analyses Spark et un assistant conversationnel (Veda), tout en maintenant une séparation claire des responsabilités (API, données, UI).

Au-delà de l'implémentation fonctionnelle, le projet met en avant une approche orientée **données et relations** : la connaissance n'est pas seulement stockée, elle est structurée dans un graphe afin de produire des recommandations explicables et des parcours de navigation cohérents. Les choix techniques (JWT/RBAC, conteneurisation, fallback JSONL) ont été intégrés dès la conception pour garantir une solution démontrable, robuste et maintenable.

Le développement de NEXORA a représenté un défi technique significatif, nécessitant l'intégration de technologies diverses et complexes : une base de données orientée graphe (Neo4j), un framework web moderne (FastAPI), une interface utilisateur réactive (React), des algorithmes d'apprentissage automatique (TF-IDF, PageRank, classification), et des outils de traitement de données à grande échelle (PySpark). La coordination de ces technologies dans une architecture cohérente a nécessité une planification rigoureuse, une communication constante au sein du binôme, et une capacité d'adaptation face aux imprévus techniques.

Un aspect particulièrement réussi du projet est la cohérence entre les différentes couches de l'architecture. Le backend expose une API bien documentée via OpenAPI, ce qui a facilité l'intégration avec le frontend. Les modèles de données sont cohérents entre les fichiers JSONL, le schéma Neo4j et les interfaces TypeScript, ce qui a réduit les erreurs

d'intégration et facilité le débogage. Les mécanismes de sécurité (JWT, RBAC) sont appliqués de manière cohérente à tous les niveaux, de l'authentification initiale jusqu'aux gardes d'autorisation sur les endpoints sensibles.

La plateforme atteint également un bon équilibre entre fonctionnalité et performance. La visualisation 3D du graphe, bien que techniquement complexe, a été optimisée pour rester responsive grâce à l'utilisation de WebGL et à la limitation de la densité des nœuds affichés. Les algorithmes de recommandation (TF-IDF, PageRank) ont été implémentés avec des mécanismes de cache pour éviter les recalculs inutiles. Le fallback JSONL permet de garantir une continuité de service même lorsque la base de données Neo4j est indisponible, ce qui est essentiel pour les démonstrations et les présentations.

TABLE 8.1 – Synthèse des objectifs et état d'implémentation

Objectif	État observé
Cartographie des compétences (graphe 3D)	Visualisation 3D WebGL intégrée
Identification d'experts	Endpoint PageRank
Recherche sémantique d'experts	TF-IDF + cosinus
Big Data (batch)	PySpark + endpoints fallback
Sécurité (JWT + RBAC)	Gardes centralisées + expiration configurable

8.2 Points Forts

Les points forts ci-dessous synthétisent les apports les plus visibles du projet, autant sur le plan technique (architecture, sécurité, performance) que sur le plan usage (navigation et accès rapide à l'expertise).

- **Graphe de connaissances natif (Neo4j)** : L'utilisation de Neo4j comme base de données principale permet de modéliser explicitement les relations entre entités (experts, compétences, projets, documents). Cette modélisation facilite les requêtes complexes comme la recherche d'experts partageant des compétences, l'identification de ponts entre départements, ou le calcul de centralité via PageRank. Contrairement à une base de données relationnelle qui nécessiterait des jointures complexes et coûteuses, Neo4j permet de traverser les relations de manière native et performante grâce à l'index-free adjacency.
- **Visualisation 3D WebGL** : La visualisation 3D du graphe constitue un élément différenciant de NEXORA. Elle permet aux utilisateurs de naviguer intuitivement dans la structure relationnelle de l'organisation, d'identifier des patterns invisibles

dans les vues tabulaires traditionnelles, et de comprendre les proximités entre experts, compétences et projets. L'utilisation de WebGL via la bibliothèque `react-force-graph-3d` permet d'afficher des milliers de nœuds avec des performances satisfaisantes sur les navigateurs modernes, tout en offrant des interactions riches (zoom, rotation, sélection de nœuds).

- **Approche IA hybride (TF-IDF, classification, PageRank, LLM avec fallback)** : NEXORA combine plusieurs approches d'intelligence artificielle pour offrir des fonctionnalités variées. La recherche sémantique d'experts utilise TF-IDF et la similarité cosinus pour trouver des profils correspondants à des requêtes en langage naturel. La classification documentaire permet d'organiser automatiquement les documents par thématique. Le PageRank identifie les experts les plus centraux dans le graphe organisationnel. L'assistant conversationnel Veda utilise un LLM (Groq) pour offrir un accès naturel à la connaissance, avec un fallback rule-based en cas d'indisponibilité du service externe.
- **Architecture API-First et conteneurisée** : L'architecture API-First de NEXORA garantit que toutes les fonctionnalités sont exposées via des endpoints REST bien documentés (OpenAPI/Swagger). Cette approche facilite l'intégration avec le frontend, permet de tester les endpoints indépendamment, et offre une base solide pour d'éventuelles intégrations avec des systèmes tiers. La conteneurisation via Docker Compose rend l'environnement reproductible et facilite le déploiement : chaque service (Neo4j, backend, frontend) s'exécute dans son propre conteneur avec ses dépendances, et l'orchestration des services est définie de manière déclarative dans le fichier `docker-compose.yml`.

8.3 Perspectives d'Amélioration

Les améliorations envisagées ne remettent pas en cause la complétude du livrable ; elles correspondent à des pistes d'industrialisation et d'enrichissement pour un contexte d'entreprise (volumétrie, supervision, sécurité avancée).

- **Industrialisation du streaming avec Kafka réel** : Le simulateur Kafka actuel démontre le concept de streaming, mais une industrialisation nécessiterait l'intégration d'un cluster Kafka réel pour gérer des flux de données en temps réel à grande échelle. Cela permettrait de capturer les événements organisationnels (nouvelles compétences acquises, nouveaux projets créés, documents publiés) et de les traiter de manière continue pour mettre à jour le graphe de connaissances et les recommandations en temps réel. L'intégration de Kafka nécessiterait également la mise en place de consumers robustes capables de gérer les réordonnements de messages et les pannes de service.

- **Renforcement de GraphRAG** : L'approche GraphRAG (Graph Retrieval-Augmented Generation) pourrait être approfondie pour améliorer les réponses du chatbot Veda. Actuellement, le système utilise une combinaison de requêtes Neo4j et de LLM, mais une implémentation plus sophistiquée pourrait inclure des embeddings de nœuds pour la recherche sémantique dans le graphe, des techniques de résumé de sous-graphes pour contextualiser les requêtes LLM, et des mécanismes de citation des sources pour améliorer l'explicabilité des réponses. Ces améliorations rendraient le chatbot plus précis et plus fiable pour des questions complexes nécessitant une compréhension fine du contexte organisationnel.
- **Amélioration de l'observabilité (métriques, traces)** : Pour un déploiement en production, il serait essentiel d'améliorer l'observabilité du système. Cela inclurait l'intégration d'outils de monitoring comme Prometheus pour collecter des métriques (latence des requêtes, taux d'erreur, utilisation des ressources), Grafana pour visualiser ces métriques en tableaux de bord, et Jaeger ou Zipkin pour le tracing distribué afin de suivre les requêtes à travers les différents services (frontend, backend, Neo4j, Spark). L'ajout de logs structurés et centralisés (via ELK Stack ou Loki) faciliterait le diagnostic des incidents et l'analyse des patterns d'utilisation.
- **Durcissement sécurité (HTTPS, audit logs)** : Bien que NEXORA implémente déjà des mécanismes de sécurité solides (JWT, RBAC, bcrypt), un déploiement en production nécessiterait un durcissement supplémentaire. Cela inclurait l'activation de HTTPS pour toutes les communications (via un reverse proxy Nginx avec TLS), l'implémentation de logs d'audit pour tracer les actions sensibles (modifications de rôles, suppressions de données), l'ajout de mécanismes de rate limiting plus sophistiqués (au-delà du login), et la mise en place de politiques de rotation des secrets (clés JWT, mots de passe). Ces mesures renforceraient la résilience du système contre les attaques et faciliteraient la conformité avec les standards de sécurité d'entreprise.

8.4 Conclusion Générale

NEXORA démontre la pertinence d'une approche par graphe pour cartographier les compétences et découvrir l'expertise interne. L'architecture proposée constitue un socle solide pour des évolutions futures vers une plateforme d'intelligence organisationnelle industrialisée.

En conclusion, ce projet a permis de concrétiser une vision ambitieuse : transformer les données RH et organisationnelles en une ressource dynamique et exploitable grâce aux technologies du graphe et de l'intelligence artificielle. La plateforme NEXORA ne se contente pas de stocker des informations sur les employés, leurs compétences et leurs projets ; elle les relie dans un réseau sémantique qui permet de découvrir des insights invisibles

dans les systèmes traditionnels : qui sont les experts connecteurs entre les départements, quelles compétences émergentes sont demandées dans les projets, comment constituer des équipes équilibrées pour un nouveau projet.

Le travail réalisé a également permis de valider l'intérêt d'une approche hybride combinant différentes technologies d'IA : les algorithmes classiques (TF-IDF, PageRank) fournissent des résultats déterministes et rapides, tandis que les LLM offrent une flexibilité et une naturalité dans les interactions. L'architecture fallback permet de garantir la continuité de service même lorsque les composants externes sont indisponibles, ce qui est essentiel pour les systèmes critiques en entreprise.

Sur le plan pédagogique, ce projet a été l'occasion de mettre en pratique de nombreux concepts de génie logiciel : architecture en couches, API-First, conteneurisation, tests unitaires, gestion de projet agile, et gestion des risques. La collaboration en binôme a permis de développer des compétences en communication, en coordination technique, et en revue de code. La documentation exhaustive (rapport, guide d'installation, glossaire) garantit que le projet peut être repris et étendu par d'autres développeurs.

Au-delà des aspects techniques, NEXORA illustre comment les technologies modernes (graph databases, LLM, Big Data) peuvent être appliquées à des problématiques concrètes d'entreprise : gestion des talents, constitution d'équipes, partage de connaissances. Dans un monde où les compétences deviennent un actif stratégique, la capacité à cartographier, valoriser et exploiter le capital de connaissances interne devient un avantage compétitif majeur. NEXORA propose une réponse technique à cette problématique, en démontrant qu'il est possible de construire une plateforme complète et fonctionnelle avec des technologies open source et des méthodes de développement agiles.

Annexe A

Guide de Démarrage Rapide

1. Cloner le dépôt du projet.
2. Créer un fichier `.env` avec les variables nécessaires.
3. Générer les données : `python data/generate_data.py -scale large`
4. Lancer la plateforme : `docker-compose up -build`
5. Accéder à l'application : `http://localhost:3000`

Bibliographie et Webographie

- [1] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems*, 30(1-7) :107-117, 1998.
- [2] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5) :513-523, 1988.
- [3] Francesco Ricci, Lior Rokach, and Bracha Shapira. *Introduction to Recommender Systems Handbook*, pages 1-35. Springer, 2011.
- [4] Neo4j Documentation. <https://neo4j.com/docs/>
- [5] Neo4j Cypher Query Language Reference. <https://neo4j.com/docs/cypher-manual/current/>
- [6] FastAPI Documentation (Sebastián Ramírez). <https://fastapi.tiangolo.com/>
- [7] Starlette : The little ASGI framework that shines. <https://www.starlette.io/>
- [8] Pydantic : Data parsing and validation using Python type hints. <https://docs.pydantic.dev/>
- [9] React Documentation. <https://react.dev/>
- [10] TypeScript : Typed JavaScript at Any Scale. <https://www.typescriptlang.org/>
- [11] Material-UI (MUI) : A comprehensive suite of UI tools to help you ship new features faster. <https://mui.com/>
- [12] JSON Web Tokens (JWT) Introduction. <https://jwt.io/introduction>
- [13] RFC 7519 : JSON Web Token (JWT). <https://datatracker.ietf.org/doc/html/rfc7519>
- [14] Apache Spark Documentation. <https://spark.apache.org/docs/>
- [15] PySpark Documentation. <https://spark.apache.org/docs/latest/api/python/>
- [16] Apache Kafka : A Distributed Streaming Platform. <https://kafka.apache.org/documentation/>
- [17] Scikit-learn : Machine Learning in Python. <https://scikit-learn.org/stable/>
- [18] Groq LPU Inference Engine Documentation. <https://groq.com/docs/>
- [19] Wayne Xin Zhao et al. A Survey of Large Language Models. *arXiv preprint arXiv :2303.18223*, 2023.

- [20] Docker Documentation. <https://docs.docker.com/>
- [21] Docker Compose Documentation. <https://docs.docker.com/compose/>
- [22] NGINX Documentation. <https://nginx.org/en/docs/>
- [23] RFC 6455 : The WebSocket Protocol. <https://datatracker.ietf.org/doc/html/rfc6455>
- [24] Ken Schwaber and Jeff Sutherland. The Scrum Guide. <https://scrumguides.org/>
- [25] OpenAPI Specification. <https://swagger.io/specification/>

Annexe B

Glossaire

Terme	Définition
ASGI	Standard Python pour applications web asynchrones
Bolt	Protocole de communication utilisé par Neo4j
Cypher	Langage déclaratif de requêtes pour Neo4j
GraphRAG	Technique combinant graphe de connaissances et LLM
JSONL	Format JSON Lines : un objet JSON par ligne
JWT	Jeton signé pour authentification stateless
LLM	Large Language Model
PageRank	Algorithme de centralité dans un graphe
RBAC	Contrôle d'accès basé sur les rôles
SPA	Single Page Application
TF-IDF	Pondération pour représentation textuelle
WebGL	API graphique pour rendu 3D dans le navigateur
WebSocket	Protocole bidirectionnel pour communication temps réel